Abstract

Efficient water management is a critical requirement in modern agriculture due to increasing water scarcity, climate variability, and the growing demand for higher crop productivity. Conventional irrigation practices often rely on manual operation or fixed scheduling, which may lead to excessive water consumption, inefficient resource utilization, and inconsistent crop growth. The integration of Internet of Things (IoT) technologies into agricultural systems offers new opportunities to address these challenges through intelligent monitoring and automated decision-making.

This work presents the design and conceptual development of an IoT-based Smart Irrigation System aimed at improving irrigation efficiency and supporting sustainable agricultural practices. The system leverages a microcontroller-based architecture combined with environmental sensing technologies to monitor key agricultural parameters in real time. By utilizing continuous data acquisition and intelligent processing, the proposed approach enables informed irrigation management that responds to changing environmental conditions.

In addition to automated monitoring, the system provides remote accessibility through a cloud-connected mobile interface, allowing users to observe field conditions and receive system feedback from any location. The proposed framework emphasizes improved water-use efficiency, enhanced soil condition awareness, and better crop management support. Overall, this project demonstrates how IoT-enabled technologies can contribute to the development of smart farming infrastructures that promote sustainability, resource optimization, and data-driven agricultural management.

Abbreviations

Abbreviations	Full Form
IoT	Internet of Things
API	Application Programming Interface
WiFi	Wireless Fidelity
SPI	Serial Peripheral Interface
I2C	Inter Integrated Circuit
UART	Universal Asynchronous Receiver Transmitter
DC	Direct Current
GPIO	General Purpose Input Output
RX/TX	Receiver/ Transmitter
ESP32	Espressif 32-bit microprocessor
PWM	Pulse Width Modulation
I2S	Inter-IC Sound

Contents

List of Tables

List of Figures

CHAPTER I

Introduction

Efficient water management has become an increasingly important challenge in modern environmental and agricultural systems. With the growing demand for food production, urban gardening, and sustainable landscape management, it is essential to develop intelligent solutions that can optimize water usage while maintaining healthy plant growth. Traditional irrigation practices often depend on manual observation or fixed watering schedules, which may not accurately reflect the real environmental needs of plants. As a result, these methods frequently lead to excessive water consumption, inconsistent soil conditions, and unnecessary human effort.

Recent technological advancements in the Internet of Things (IoT) have created new possibilities for improving environmental monitoring and automated control systems. IoT technology enables the integration of sensors, microcontrollers, and wireless communication networks to collect and transmit real-time data from physical environments. By continuously monitoring parameters such as soil conditions, temperature, humidity, and rainfall, IoT-based systems can support intelligent decision-making and adaptive control strategies.

Smart irrigation systems represent one of the most practical applications of IoT technology in plant management. These systems can be used in agricultural fields, greenhouses, nurseries, rooftop gardens, and home gardening environments where maintaining proper soil moisture and environmental balance is essential for plant health. By automatically adjusting irrigation based on real-time conditions, such systems help reduce water wastage while ensuring that plants receive the appropriate amount of water required for growth.

In addition to improving irrigation efficiency, modern smart irrigation solutions can also provide remote monitoring capabilities through cloud-connected platforms and mobile applications. This allows users to observe environmental conditions, receive alerts, and manage irrigation systems from remote locations. Such connectivity improves convenience, reduces manual intervention, and enhances the reliability of irrigation management.

The proposed system integrates environmental sensing, automated control, and digital connectivity to support efficient irrigation management. By combining monitoring technologies with intelligent control strategies, the system aims to improve water-use efficiency, support healthier plant growth, and contribute to more sustainable plant management practices.

Motivation of The Work

The increasing scarcity of freshwater resources and the growing demand for sustainable agriculture have created a strong need for smarter irrigation practices. The primary motivation behind this project is to promote efficient water usage and minimize unnecessary wastage in agricultural activities.

Another major motivation is to support farmers with a simple, reliable, and user-friendly irrigation system that can make farming more convenient and less dependent on constant manual supervision. The project aims to encourage smarter agricultural practices that are easy to understand and accessible even to users with limited technical knowledge.

The work is also motivated by the need to improve plant health and crop productivity through better awareness of soil conditions. Understanding important parameters such as soil moisture and soil pH can help farmers make better decisions regarding irrigation and crop selection.

In addition, the project focuses on creating an eco-friendly and efficient agricultural environment that supports sustainable farming for the future. The overall vision of this work is to contribute toward modern, intelligent, and resource-conscious agriculture that benefits both farmers and the environment.

Objective

To design and develop an **intelligent IoT-based smart irrigation system** that optimizes **water usage**, enhances **soil health**, and maximizes **crop productivity** through **real-time monitoring**, **crop-specific automation**, and **data-driven decision-making**.

Specific Objectives

- To continuously monitor critical **environmental parameters** such as **soil moisture**, **soil pH**, **temperature**, **humidity**, **rainfall**, and **water tank level** using **integrated sensors**.
- To automate irrigation using an **ESP32 microcontroller** by making **real-time decisions** based on **sensor data**, thereby reducing **manual intervention** and **human error**.
- To implement **crop-specific irrigation strategies** by dynamically adjusting **moisture thresholds**, **irrigation duration**, and **environmental conditions** for different crops (e.g., rice, wheat, vegetables).
- To enhance **water-use efficiency** by integrating **rain detection mechanisms** and **weather forecast APIs**, preventing **unnecessary irrigation** during or before rainfall.
- To dynamically regulate **irrigation duration** based on **temperature** and **humidity variations** to compensate for **evaporation** and **transpiration losses**.
- To adapt irrigation control based on **soil pH classification** (**acidic**, **neutral**, **alkaline**) for improved **soil condition** and **nutrient availability**.
- To provide **intelligent fertilizer recommendations** (type, quantity, and brand) based on **soil pH analysis** through the **IoT-based user interface**.
- To enable **real-time monitoring**, **alerts**, and **remote control** via a **mobile application (Blynk platform)**.
- To incorporate **data logging** and **predictive analytics** for future **irrigation planning** and improved **decision-making**.

CHAPTER II

Literature Survey

1. **Automatic Irrigation System by 555 IC** [International Journal of Advances in Engineering and Management (IJAEM) Volume 6, Issue 01 Jan 2024, pp: 598-602 www.ijaem.net ISSN: 2395-5252]

   Research on automated irrigation has focused on improving water efficiency through sensor-based control systems. Earlier timer-based systems lacked accuracy because they could not detect actual soil moisture. Nabaneeta Banerjee et al. (2024) proposed a low-cost irrigation system using a 555 timer IC with a soil moisture sensor. The pump activates only when moisture falls below a set threshold, ensuring better control than fixed-time irrigation. The 555 IC in astable mode generates timing pulses for reliable switching. Compared to microcontroller systems, this design is cheaper and suitable for small farms while still ensuring efficient watering. Future improvements suggested include IoT integration, solar power, and multi-parameter sensing. The system lacks data processing and remote monitoring capabilities, depends on manual circuit adjustments, and uses resistive sensors that may degrade over time.

2. **Automatic Irrigation System using Moisture Sensor** [International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering, Vol. 9, DOI:10.17148/IJIREEICE.2021.9557, 05-05-2021]

   Automatic irrigation systems increasingly use soil-moisture sensing to reduce water wastage. Mishra et al. (2021) developed a closed-loop system where a moisture sensor sends data to an Arduino controller. When moisture drops below a threshold, the pump starts automatically. Resistive probes estimate water content through soil conductivity, offering low-cost automation. Earlier timer systems often caused overwatering, while sensor-based systems respond accurately to soil needs. Other studies have also used wireless networks, GPRS, and IoT for remote monitoring and better irrigation scheduling. Overall, the paper supports water conservation, reduced labour, and improved crop productivity. The system depends on resistive moisture sensors that degrade over time, lacks remote monitoring functions, and does not integrate weather or environmental data, limiting its adaptability under varying field conditions.

3. **An IoT-based Smart Irrigation System using the Blynk Application** [Chetan Naik J. et al. link-https://share.google/UDg81FZO1AUnlOKFH]

   Chetan Naik J. et al. in *"An IoT-based Smart Irrigation System using the Blynk Application"* (2024) proposed a smart irrigation system using ESP32, soil moisture sensors, DHT11 sensors, relays, solenoid valves, and the Blynk mobile application for remote monitoring and control. The system monitored soil moisture, temperature, humidity, and water level conditions through wireless sensor networks and transmitted the data to the Blynk cloud using Wi-Fi. Automatic irrigation was achieved by controlling the solenoid valve based on soil moisture threshold values.

The system also provided SMS notifications and smartphone-based control for farmers. The study highlighted advantages such as water conservation, reduced manual labour, improved crop monitoring, and real-time agricultural management. However, the system mainly focused on moisture-based irrigation and basic IoT monitoring. It lacked advanced functionalities such as crop-specific irrigation logic, weather API integration, soil pH analysis, offline operational capability, intelligent fault detection, and detailed water usage analytics. The proposed project improves upon these limitations by integrating weather prediction, crop customization, pH monitoring, offline mode, and advanced smart irrigation analytics into a unified IoT-based irrigation platform.

4. **Smart Irrigation System Using Internet of Things** [2019 IEEE 9th International Conference on System Engineering and Technology (ICSET), 7 October 2019, Shah Alam, Malaysia]

   Nor Adni Mat Leh et al. in *"Smart Irrigation System Using Internet of Things"* presented an IoT-based smart irrigation system using Arduino Mega 2560, ESP8266 Wi-Fi module, soil moisture sensors, DHT11 sensors, water pumps, and the Blynk application for real-time monitoring and control. The system automatically irrigated plants based on soil moisture conditions and allowed users to monitor temperature, humidity, and soil moisture values through smartphones connected to the internet. The research compared smart irrigation with conventional irrigation methods and observed that smart irrigation improved plant growth, reduced overwatering, and maintained proper soil moisture conditions. Additional experiments were conducted under different sunlight conditions, pH values, and windy environments to analyze plant performance. The study concluded that smart irrigation systems are more efficient and environmentally beneficial compared to traditional irrigation techniques. However, the system required multiple sensors and pumps for large-scale agricultural fields, making implementation costly. It also lacked advanced features such as weather forecasting, crop-specific irrigation logic, offline operational capability, intelligent fault detection, and detailed water usage analytics. The proposed project improves upon these limitations by integrating weather prediction, crop customization, offline mode, and advanced IoT-based monitoring into a unified smart irrigation platform.

CHAPTER III

Theoretical background

Internet of Things (IoT) in Smart Irrigation: The Internet of Things (IoT) refers to a network of interconnected devices capable of sensing, processing, and transmitting data over the internet. In smart irrigation and plant management systems, IoT enables continuous monitoring of environmental parameters and facilitates automated control based on real-time data. By integrating sensors with microcontrollers and cloud platforms, IoT-based systems transform conventional irrigation into an intelligent, data-driven process.

Sensor-Based Environmental Monitoring: Efficient irrigation depends on accurate measurement of environmental and soil conditions. Sensors such as soil moisture, temperature, humidity, rainfall, and pH sensors provide critical inputs that reflect the actual needs of plants. These sensors form a closed-loop control system, where real-time data is continuously monitored and used to adjust irrigation decisions. This approach minimizes water wastage and ensures optimal plant growth conditions.

Soil Moisture and Irrigation Control: Soil moisture is the primary parameter for determining irrigation requirements. Moisture sensors estimate the water content in soil, enabling the system to identify when irrigation is necessary. Unlike traditional timer-based systems, moisture-based control ensures that water is supplied only when required, preventing both under-irrigation and over-irrigation.

Environmental Factors and Evapotranspiration: Temperature and humidity significantly influence plant water requirements through the process of evapotranspiration, which includes water evaporation from soil and transpiration from plant surfaces. Higher temperatures and lower humidity increase water loss, requiring adjustments in irrigation. Monitoring these parameters allows the system to dynamically adapt irrigation duration and frequency.

Soil pH and Nutrient Availability: Soil pH is a crucial factor affecting plant growth, as it determines the availability of essential nutrients. Different crops require specific pH ranges for optimal nutrient absorption. Monitoring soil pH helps in maintaining suitable soil conditions and supports better crop health. It also provides a basis for recommending appropriate fertilizers.

Rain Detection and Predictive Irrigation: Rainfall is an important natural factor that influences irrigation requirements. Real-time rain detection combined with weather forecasting enables predictive irrigation, where watering can be delayed or skipped if rainfall is detected or expected (ref no.5). This approach improves water conservation and enhances system efficiency.

Cloud-Based Monitoring and Control: Cloud platforms play a vital role in IoT systems by enabling remote access, data visualization, and user interaction. Sensor data transmitted to the cloud can be displayed through mobile applications, allowing users to monitor environmental conditions in real time. Cloud integration also enables alert systems and remote-control functionalities, enhancing usability and system responsiveness.

Component Description

1. ESP32 Microcontroller:

The ESP32-WROOM-32 is a low-cost, high-performance microcontroller with integrated Wi-Fi (802.11 b/g/n) and Bluetooth (v4.2) capabilities, making it ideal for IoT applications. It features a dual-core Tensilica Xtensa LX6 processor with a clock speed of up to 240 MHz and supports multiple GPIO interfaces for sensor integration.

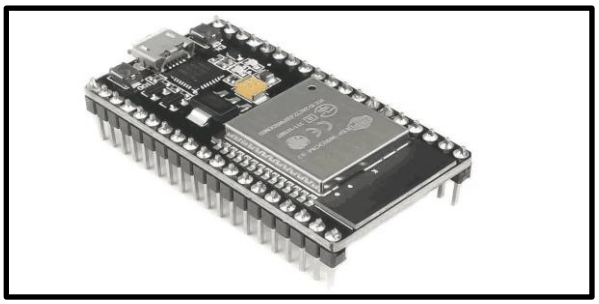

Fig 3.1: ESP32

Key Specifications:

- Microcontroller: Tensilica Xtensa LX6 (Dual-Core)
- Operating Voltage: 3.0V – 3.6V
- Input Voltage: 5V (via VIN/USB)
- Digital I/O Pins: ~30 (varies by board)
- Analog Input Pins: 12-bit ADC (up to 18 channels)
- DAC Pins: 2 (8-bit resolution)
- Flash Memory: 4 MB (external)
- SRAM: 520 KB
- EEPROM: Not dedicated (emulated in flash)
- Clock Speed: Up to 240 MHz
- Wi-Fi: 802.11 b/g/n
- PWM Channels: 16
- Communication Protocols: UART, SPI, I2C, I2S

The ESP32 acts as the central control unit of the smart irrigation system. It continuously collects data from all connected sensors such as soil moisture, temperature, humidity, pH, rain, and water level. Based on these inputs, it processes the data and makes real-time decisions to control the water pump for efficient irrigation.

In addition, the ESP32 enables IoT connectivity by transmitting sensor data to the Blynk cloud platform, allowing users to monitor conditions and receive alerts remotely. It also integrates environmental inputs (like weather data) to support adaptive and predictive irrigation control. Overall, it manages data processing, automation, and communication, ensuring reliable and intelligent system operation. (***see Ref. 20***)

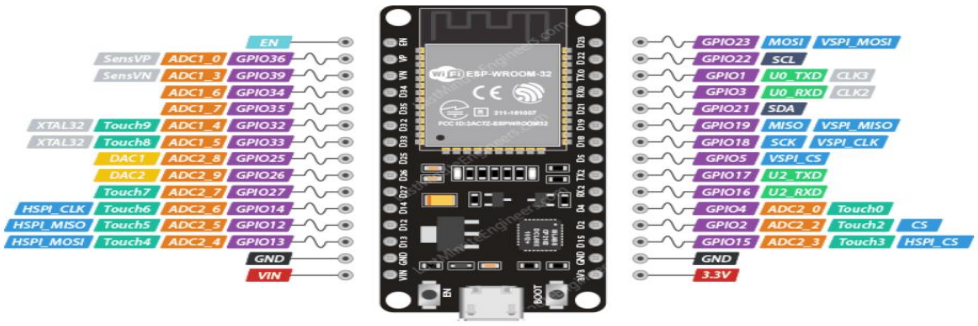

Fig 3.2: ESP32 Pin Diagram

2. Capacitive Soil Moisture Sensor (v1.2)

The Capacitive Soil Moisture Sensor measures soil water content using capacitive sensing. It detects changes in soil properties and outputs an analog signal, offering better durability and stability compared to resistive sensors as it avoids corrosion. (***see Ref.21***)

Key Specifications:

- Operating Voltage: 3.3V – 5V
- Output: Analog
- Type: Capacitive (non-corrosive)

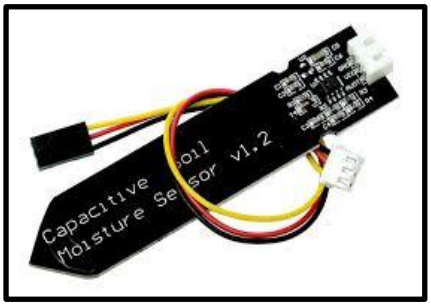

Fig 3.3: Soil Moisture Sensor

The sensor measures soil moisture based on dielectric changes and sends data to the ESP32. It helps in automatic irrigation control by triggering the pump when soil becomes dry, ensuring efficient water usage and better plant growth.

3. Temperature and Humidity Sensor (DHT11):

The DHT11 is a basic digital sensor used for measuring ambient temperature and humidity. It uses a capacitive humidity sensor and a thermistor to provide calibrated digital output. (***see Ref.17***)

Key Specifications:

- Operating Voltage: 3.3V – 5V
- Temperature Range: 0°C to 50°C (±2°C accuracy)
- Humidity Range: 20% – 90% RH (±5% accuracy)

It provides environmental data required for adaptive irrigation control.

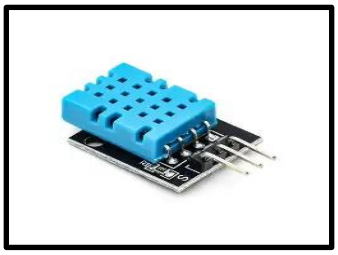

Fig 3.4: Temperature and Humidity

4. **Ultrasonic Sensor (HC-SR04):**

The **HC-SR04 ultrasonic sensor** is used for non-contact distance measurement to determine water level in the tank. It operates by transmitting ultrasonic pulses and measuring echo return time. (***see Ref.18***)

Key Specifications:

- Operating Voltage: 5V
- Measuring Range: 2 cm – 400 cm
- Accuracy: ±3 mm
- Operating Frequency: 40 kHz

It ensures safe operation by monitoring tank water levels.

Fig 3.5: Ultrasonic Sensor

5. **Rain Sensor Module (YL-83):**

The rain sensor module (YL-83) detects rainfall based on changes in electrical conductivity across its surface. (***see Ref. 19***)

Key Specifications:

- Operating Voltage: 3.3V – 5V
- Output: Analog and Digital
- Detection Method: Resistive plate sensing

It enables the system to pause irrigation during rainfall conditions.

Fig 3.6: Rain sensor

6. **soil pH sensor:**

The analog pH sensor kit measures soil acidity or alkalinity by detecting hydrogen ion concentration in the soil solution. (***see Ref. 21***)

Key Specifications:

- Operating Voltage: 5V (sometimes 9V for stable reference)
- pH Range: 0 – 14
- Accuracy: ±0.01 to ±0.1 pH (depending on calibration)
- Output: Analog voltage

It helps in assessing soil condition and supports fertilizer recommendation logic.

Fig 3.7: soil pH sensor

7. **Motor Driver (L298N):**

The **L298N motor driver module** is used to control the DC pump by amplifying control signals from the ESP32. It allows bidirectional control of motors using an H-bridge configuration.

(***see Ref.22***)

Key Specifications:

- Operating Voltage: 5V – 35V
- Output Current: Up to 2A per channel
- Logic Voltage: 5V
- Channels: Dual H-Bridge

It acts as an interface between low-power control signals and high-power devices.

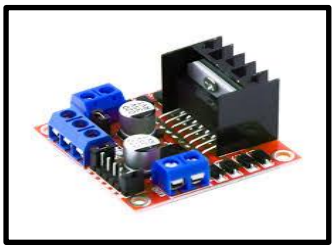

Fig 3.8: Motor Driver

8. **DC Water Pump (5V Submersible Pump) :**

The **DC water pump** is responsible for supplying water to the plants. It operates based on signals received from the motor driver.

Key Specifications:

- Operating Voltage: 5V or 12V DC
- Flow Rate: ~80–120 L/h (depending on model)
- Current Consumption: 300–500 mA
- Type: Submersible mini pump

It performs the physical irrigation process.

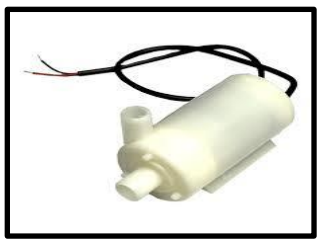

Fig 3.9: DC water pump

9. **Buck Converter (LM2596 Step-Down Module):**

The **LM2596 Buck Converter** is a DC-DC step-down voltage regulator used to convert a higher input voltage into a stable lower output voltage efficiently. It helps provide suitable voltage levels to low-power electronic components while minimizing power loss. (***see Ref.23***)

Key Specifications:

- **Model:** LM2596
- **Input Voltage:** 4V – 40V
- **Output Voltage:** 1.25V – 35V (adjustable)
- **Maximum Output Current:** 3A
- **Efficiency:** Up to 92%

It ensured efficient power management and protected sensitive components from voltage fluctuations.

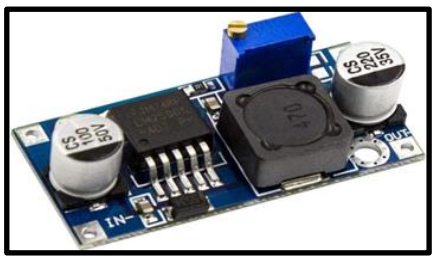

Fig 3.10: Buck converter

10. **Breadboard**

The **breadboard** is a reusable prototyping platform used for assembling and testing electronic circuits without soldering. It contains interconnected conductive strips that allow components and jumper wires to be connected easily, making circuit modification and troubleshooting convenient during development.

Key Specifications:

- **Type:** Solderless breadboard
- **Power Rails:** Positive and negative supply lines
- **Connection Method:** Internal metal strip connections
- **Reusable:** Yes

The breadboard was used during the prototyping and testing phase to connect the ESP32, sensors, motor driver, and other electronic components. It enabled easy circuit modification, debugging, and component integration before final implementation.

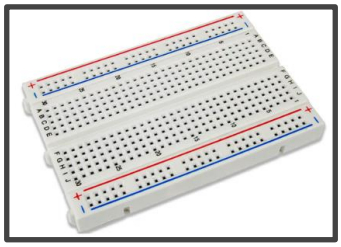

Fig 3.11: Breadboard

CHAPTER IV

Step By Step Implementation

This project presents a smart IoT-based irrigation system that automates watering decisions using real-time environmental and soil data. It integrates sensing, prediction, and control to optimize water usage, improve crop health, and ensure reliable operation under both online and offline conditions.

Step 1: Initial Irrigation System Implementation

Choosing Components: The strategic selection of hardware components for an IoT-based smart irrigation system is governed by the necessity for real-time data precision, robust power management, and seamless cloud connectivity. At the core of this architecture is the ESP32 microcontroller, selected for its integrated Wi-Fi and Bluetooth capabilities, which facilitate high-speed telemetry and remote monitoring. To ensure comprehensive environmental awareness, the system integrates a diverse sensor suite: the DHT11 and soil moisture sensors provide critical atmospheric and subterranean humidity data, while the HC-SR04 sonar sensor and rain sensor monitor water reservoir levels and precipitation status, respectively.

A primary design challenge involves managing multiple voltage domains to maintain sensor accuracy and motor efficiency. A 12V DC power supply serves as the primary source, driving the L298N motor driver, which provides the necessary current amplification to operate the DC pump motor. Given the sensitivity of the pH sensor, a dedicated 9V power supply is utilized to ensure a stable reference voltage for soil acidity monitoring. To bridge the gap between high-voltage supplies and the logic-level requirements of the microcontroller and sensors, a Buck Converter is employed for efficient 9V to 5V step-down conversion, minimizing heat dissipation compared to linear regulators. This modular approach to component selection ensures a resilient, scalable, and energy-efficient solution for automated precision agriculture.

Step 2: Components Checking/ Testing,

The assembly and validation of the smart irrigation system follow a modular "unit-testing" philosophy to ensure reliability before final integration. Each component is first verified in isolation: the ESP32 is flashed with a basic blink sketch to confirm logic health, while the DHT11, soil moisture, and rain sensors are tested using serial monitor outputs to validate data accuracy. The sonar sensor is calibrated by measuring known distances to ensure the reservoir depth readings are precise. Simultaneously, the power hardware undergoes rigorous checks; the 12V DC supply is verified for the L298N motor driver, and the Buck Converter is meticulously tuned using a multimeter to ensure the 9V to 5V step-down is stable, preventing over voltage damage to the microcontroller. The pH sensor is calibrated using buffer solutions to ensure the 9V supply provides the necessary clean reference voltage for electrochemical sensing.

Once individual stability is confirmed, the components are interconnected to form the comprehensive circuit. Jumper wires are utilised for the breadboard prototyping phase and for connecting the ESP32's GPIO pins to the low-current sensor leads. For the high-

current demands of the DC pump motor and the primary 12V power rails, standard insulated copper wires are used to reduce resistance and prevent overheating. The L298N driver acts as the bridge, linking the ESP32's digital logic signals to the high-power pump circuit. This transition from individual modules to a unified system allows for systematic troubleshooting, ensuring that when the final circuit is energized, the sensors accurately trigger the pump through the ESP32's central logic without electrical interference or signal degradation.

Advantages of Components:
By using several different sensors, an IoT irrigation system changes from a basic timer into a smart tool that makes decisions based on real data. This setup creates a "closed-loop," meaning the system constantly checks the environment and adjusts itself. Instead of just watering at a set time, it only uses water and fertilizer when the plants actually need them, which saves resources and keeps the plants healthier.

Step 3: Integration of Soil pH Monitoring

The next enhancement involved adding the pH sensor to monitor soil acidity or alkalinity. Initially, this sensor was used only for displaying pH values and did not influence the irrigation logic. However, this step was important as it introduced soil chemistry into the system, laying the groundwork for future intelligent decision-making related to crop health and nutrient management.

Step 4: Environmental Awareness using Rain Sensor and Weather API
The system was then upgraded to include a rain sensor and weather API integration, making it more environmentally aware. The rain sensor provided real-time rainfall detection, while the weather API enabled prediction of upcoming rainfall. This allowed the system to move beyond reactive irrigation and incorporate predictive control. Irrigation could now be skipped or delayed if rain was detected or expected, improving water efficiency significantly. (***see Ref no.5***)

Environmental and Substrate Monitoring:

The combination of atmospheric and soil sensors ensures the system reacts to the actual needs of the plant rather than environmental assumptions.

- **Soil Moisture Sensor:** This is the primary trigger for the system. It prevents both under watering (which causes wilting) and over watering (which leads to root rot). By maintaining moisture within a specific "sweet spot," we ensure optimal nutrient uptake.
- **DHT11 (Temperature & Humidity):** These parameters dictate the evapotranspiration rate. On a hot, dry day, the plant loses water faster; the DHT11 allows the ESP32 to preemptively adjust irrigation cycles before the soil even dries out.
- **pH Sensor:** Soil acidity directly affects a plant's ability to absorb minerals. Monitoring pH ensures that the water or fertiliser being added isn't making the

soil too alkaline or acidic, which could lock out essential nutrients like nitrogen or phosphorus.

- The system integrates a Weather API to fetch localized meteorological data, enabling proactive irrigation management based on precipitation probability. By combining these forecasts with real-time rain sensor detection, the ESP32 automatically disables the motor to maximize water conservation.

Step 5: Intelligent Irrigation Control Logic

At this stage, the irrigation logic was significantly improved by combining multiple parameters such as soil moisture, temperature, humidity, pH, and rainfall conditions. The system began deciding not only when to irrigate but also how long to irrigate. Pump runtime became dynamic, adjusting based on environmental conditions like high temperature or low humidity. Emergency conditions, such as critically low moisture, were also handled with override logic to ensure plant safety.

Step 6: Water Conservation and Analytics

A water-saving mechanism was introduced to quantify the efficiency of the system. Whenever irrigation was skipped due to rainfall, the system calculated the amount of water saved based on pump flow rate and runtime. This value was accumulated over time, providing insight into total water conservation. This feature transformed the system from simple automation into a data-driven solution focused on resource management.

Resource Management and System Resilience

The system extends beyond basic monitoring by incorporating intelligent water analytics and a proactive hardware safety layer:

- Volumetric Water Tracking: The system estimates water savings by calculating the volume of water that would have been consumed during scheduled irrigation cycle, but was instead conserved due to detected rainfall. This estimation relies on two primary variables: the pump flow rate and the dynamically adjusted pump runtime.

Mathematical Formula

To determine the total volume of water saved (in liters), the system converts the flow rate and runtime into consistent units (seconds) before multiplying them:

$$\text{Water Saved (L)} = \left(\frac{\text{pumpFlowRate (L/min)}}{60}\right) \times \left(\frac{\text{pumpTime (ms)}}{1000}\right)$$

(4.1)

Operational Logic ***(see Ref no.6)***

The calculation is triggered when the following conditions are met:

1. Moisture Threshold: Soil moisture falls below the required limit, which would typically activate the pump.
2. Rainfall Detection: Rainfall is either currently detected or has recently occurred, overriding the irrigation trigger.

Step 7: Fertilizer Recommendation System

Using the pH data, the system was enhanced to provide fertilizer recommendations. Based on the detected soil condition, appropriate fertilizers along with their quantities were suggested. This allowed the system to support not only irrigation but also soil nutrient management, ensuring better plant growth and healthier crops. (***see Ref no. 9***)

Step 8: Crop-Based Customization

To make the system adaptable to different agricultural needs, crop selection functionality was added. Each crop has different water requirements, so selecting a crop automatically adjusts parameters like moisture threshold and irrigation duration. This ensures that the system provides tailored irrigation based on the specific crop being grown. (***see Ref no. 10***)

Step 9: Fault Detection and Safety Mechanisms

A comprehensive fault detection system was implemented to ensure safe and reliable operation. The system continuously monitors all sensors and identifies abnormal readings or failures. It also includes protection against conditions like pump dry run and sensor faults. When a fault is detected, appropriate actions are taken, such as stopping the pump and notifying the user, ensuring both hardware safety and system reliability.

Step 10: Offline Capability and System Resilience

The system is designed with a resilient connectivity hierarchy that prioritizes local hardware functionality even during network fluctuations. While the ESP32 seeks a Wi-Fi connection upon startup to initialize time-sensitive features like the weather API, the system is not paralyzed by a lack of internet. If the device operates in offline mode, the core irrigation logic remains fully active (***see Ref no. 7***); the sensors continue to monitor the environment, and the circuit triggers the pump based on local moisture levels. In this state, only the external API calls and Blynk cloud updates are suspended.

The circuit continuously scans for available Wi-Fi in the background without interrupting the main operational loop. As soon as a connection is restored, the system automatically reactivates the weather API and re-establishes the link with the Blynk platform, instantly resuming real-time data streaming and cloud-based monitoring.

Connectivity Logic Overview

Component	Online Status	Offline Status
Local Sensors	Active (Streaming Data)	Active (Local Control)
Irrigation Pump	Driven by Cloud + Local Data	Driven by Local Thresholds
Weather API	Active (Forecast Updates)	Inactive (Uses Sensor Data)
Blynk Dashboard	Real-time Sync	Static (Last Known State)

Table 4.1: Differences between Online and Offline Mode

A crucial distinction is made between internet access and Blynk platform synchronization: if the ESP32 is successfully connected to the internet but the Blynk cloud remains unreachable due to server maintenance or authentication delays the irrigation circuit continues to function as usual. The local logic, including soil moisture monitoring, ultrasonic tank calculations, and sensor fault detection, remains fully active. In this state, the only limitation is that real-time telemetry and dashboard updates are suspended. As soon as the Blynk platform regains access, the data link is automatically restored, and the accumulated telemetry is once again visualized on the mobile interface.

Step 11: User Interaction via Blynk Platform

The integration of the Blynk platform enabled real-time monitoring and user interaction. Sensor data such as temperature, humidity, soil moisture, and water level were displayed on a mobile dashboard. Additional features like fertilizer recommendations, contributor display, and water-saving graphs were also implemented. The platform provided remote access and notifications, making the system more user-friendly and interactive. (***see Ref. 3***)

Enhancement in Blynk Platform:

1. **Network Initialization:** The process begins as the ESP32 utilizes its onboard Wi-Fi to establish a continuous, bidirectional link with the Blynk cloud server, creating the primary bridge between field hardware and the user.
2. **Data Acquisition and Processing:** Once connected, the system captures raw data from the soil moisture, DHT11, and rain sensors. This information is processed locally by the microcontroller to prepare it for cloud transmission.
3. **Real-Time Telemetry Streaming:** The processed data is then streamed to the Blynk cloud, allowing for instantaneous updates to the customized mobile dashboard regardless of the user's location.
4. **Visual Data Representation:** Upon reaching the dashboard, the telemetry is transformed into dynamic gauges and historical graphs, providing a clear and comprehensive overview of the micro-climate surrounding the plants.

5. **System Initialization and pH Baseline:** Upon startup, the system immediately initiates a pH sensor reading. This establishes the baseline soil chemistry, which is instantly transmitted to the Blynk dashboard to provide the user with immediate visibility into the soil's health.

   Rather than serving as a static measurement, the pH reading actively drives the system's logic: the soil moisture threshold automatically adjusts based on the soil's acidity or alkalinity, optimizing irrigation for the specific soil type. (***see Ref. 14,15,16***) Simultaneously, The ESP32 processes the pH data and sends fertilizer recommendations to the Blynk dashboard, suggesting precise amendments tailored to the detected pH level. This integrated approach ensures that both water delivery and nutrient management are precisely synchronized with the real-time chemical profile of the environment.

6. **Water Level Assessment:** The system utilizes sonar sensor data to calculate the reservoir's volume. This is represented on the app as a live percentage, providing a clear visual of available resources.
7. **Environmental Monitoring and Visualization:** Data from field-level sensors is continuously streamed to the dashboard, where it is visualized through gauges and graphs to monitor the micro-climate and soil conditions in real-time.
8. **Safety Logic and Notifications:** The platform continuously monitors for critical conditions with a specific and proper time duration. If the water level drops below a safe threshold, the system triggers an immediate push notification and safety lockout to prevent pump damage, ensuring a fail-safe environment through constant feedback.

Faced Challenges and overcoming the errors

The transition from individual component testing to a fully integrated IoT system introduced several complex troubleshooting challenges. A major issue was the integration paradox, where components operated correctly in isolation but failed when connected together. This was primarily caused by power bus contention and signal interference; the high current drawn by the L298N motor driver during pump operation generated voltage fluctuations that caused ESP32 brownouts and unstable pH sensor readings. Firmware uploading also became problematic because sensors connected to RX/TX and boot-strapping GPIO pins interfered with UART communication, requiring temporary disconnection of jumper wires during every code upload.

Software and hardware compatibility issues further complicated development. Early Arduino IDE board drivers frequently conflicted with CP210x and CH340 USB-to-UART controllers, resulting in "Port Not Found" errors and IDE instability while handling the Blynk library. Additionally, different ESP32 development boards showed inconsistent behavior, including varying boot requirements and differing I2C/SPI pin mappings, leading to reboot loops and unexpected crashes during testing.

To achieve a stable and synchronized smart irrigation system, the following corrective measures were implemented:

● **Driver Stabilization:** Installed legacy-compatible drivers to ensure reliable communication between the PC and ESP32.

● **GPIO Re-mapping:** Reassigned GPIO pins to avoid reserved boot-strapping pins and startup conflicts.

● **Signal Integrity:** Implemented a common-ground architecture and isolated motor power from control circuitry to reduce electrical noise and data interference.

Hardware vs. Software Challenges

To overcome the technical bottlenecks during integration, an isolation-first approach was adopted. The ESP32 firmware was uploaded separately before connecting it to the main circuit, eliminating interference from boot-strapping pins and ensuring stable serial communication through a reliable micro-USB data cable. Driver-related instability was resolved by switching to more compatible development drivers, which removed recurring COM port timeout issues.

System reliability was further improved by upgrading to high-quality ESP32 modules with better power handling and PCB design. Since unstable power distribution caused frequent crashes, the power architecture was redesigned using separate supply paths: a Buck Converter provided regulated 5V power to the sensors, while the L298N motor driver used an independent supply. This isolated motor noise from the sensitive control circuitry. Common grounding and high-quality data cables minimized signal interference and communication failures, resulting in a stable and reliable smart irrigation system.

Challenge	Solution
Driver Conflicts	Updated to stable version of controller and module drivers.
Port Recognition	Standardized the connection protocol and boot sequence.
System Crashes	Optimized code to handle the large Blynk library and fixed pin assignments.
Signal Noise	Established a common ground to prevent electrical interference.

Table 4.2: Challenges faced

Working Flow Chart

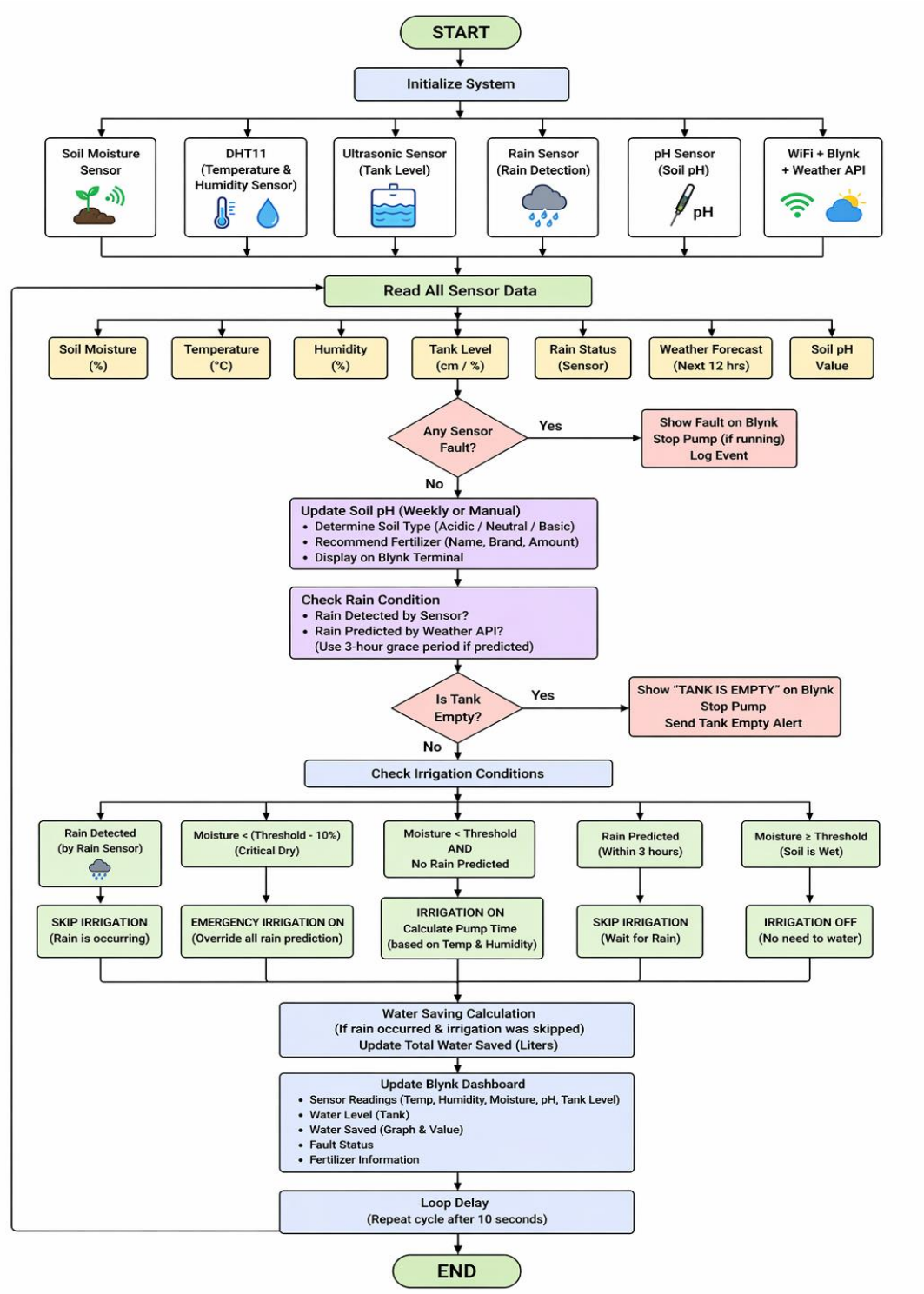

Fig 4.1: Flow Chart

Block Diagram

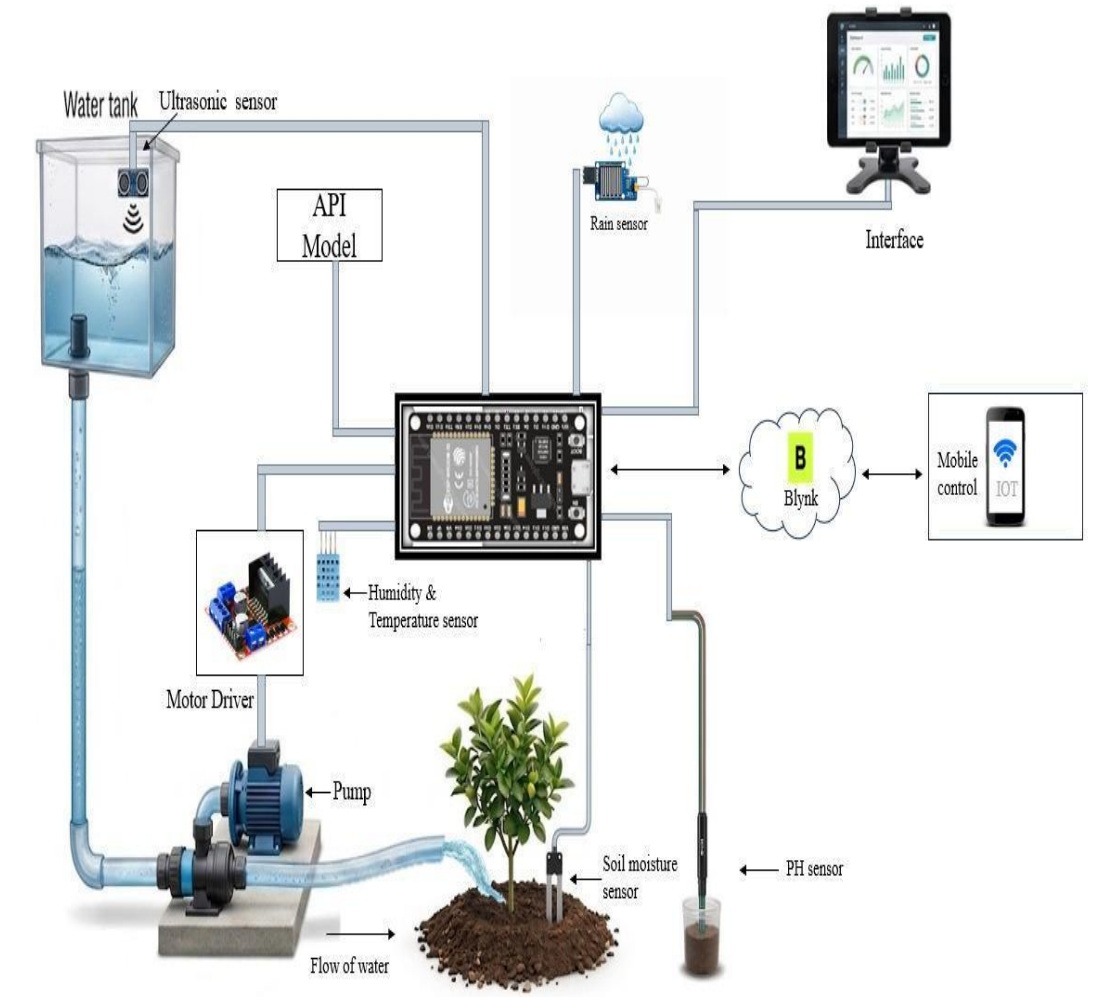

Fig 4.2 Block Diagram

Circuit Diagram

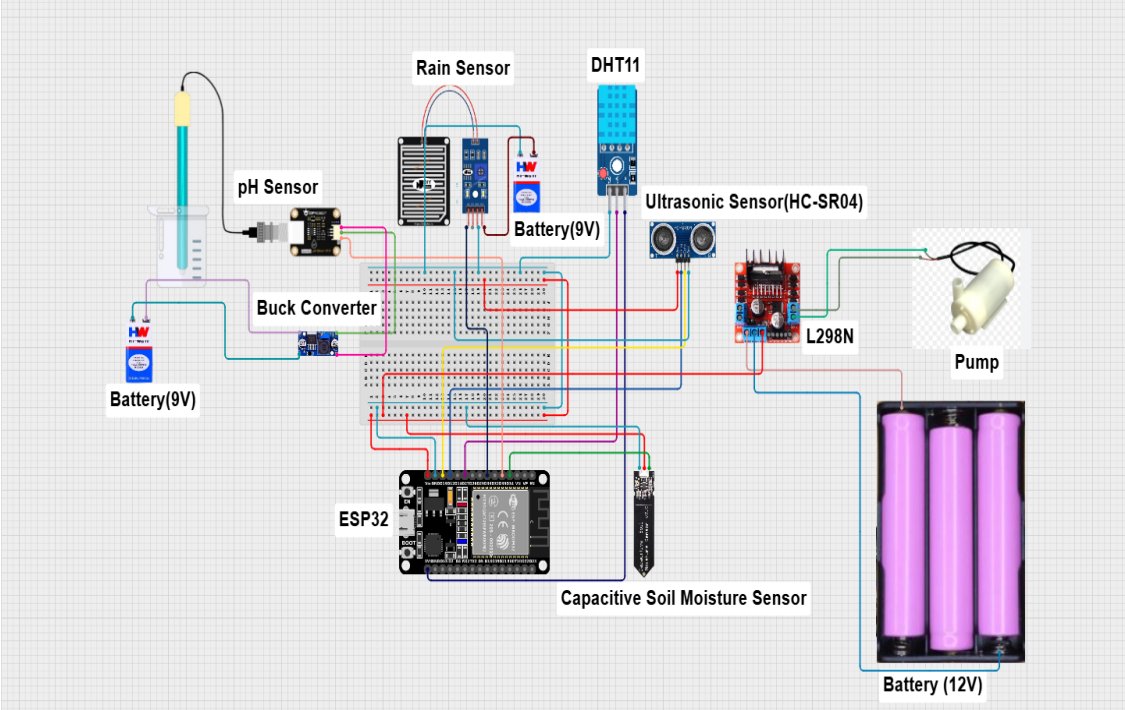

Fig 4.3: Circuit Diagram

Wiring Connection

DHT11 Sensor • VCC → 3.3V • GND → GND • DATA → pin 27 Ultrasonic Sensor • TRIG → pin 12 • ECHO → pin 13 • VCC → 5V • GND → GND Rain Sensor • A0 → pin 33	Soil Moisture Sensor • VCC → 3.3V • GND → GND • A0 → pin 34 • D0 → not connected pH Sensor • VCC → 5V • GND → GND • P0 → pin 35 Pump • Control input → pin 32

Final Code

```
/********************** DEFINES & GLOBAL VARIABLES **********************/
#define BLYNK_TEMPLATE_ID "TMPL3pzSYueE5"
#define BLYNK_TEMPLATE_NAME "Iot plant irrigation "
#define BLYNK_AUTH_TOKEN "zY1k_9SLEGHwoLg-qNOG0iXA06_7N1VC"
bool contributorsShown = false; // All flags are by default false
bool fertilizerShownOnce = false;
bool fertilizerCalculated = false;
bool soilSensorFault = false;
bool dhtFault = false;
bool ultrasonicFault = false;
bool phSensorFault = false;
bool pumpFault = false;
bool rainSensorFault = false;
bool offlineMode = false;
unsigned long lastReconnectAttempt = 0;
const unsigned long RECONNECT_INTERVAL = 10000; // 10 sec
String fertilizerName = "Not calculated";
String fertilizerBrand = "N/A";
String selectedCrop = "Others";
float fertilizerAmount = 0.0;   // kg per acre
bool tankEmptyLatched = false;
bool rainEventLatched = false;
bool soilWasWet = true;// TO STOP PUMP TRIGGERING REPEATEDLY
unsigned long rainPredictionStartTime = 0;
const unsigned long RAIN_GRACE_PERIOD = 3 * 60 * 60 * 1000; // 3 hours
int CRITICAL_MOISTURE_OFFSET = 10; // % below threshold
unsigned long lastWeatherCheck = 0;
const unsigned long WEATHER_INTERVAL = 2 * 60 * 60 * 1000; // 2 hours // MAX
1000 CALLS PER DAY
bool rainPredicted = false;
bool rainConfirmed = false;
unsigned long lastLoopTime = 0;
const unsigned long LOOP_INTERVAL = 10000;// 10 seconds
#define RAIN_AO_PIN 33
#define RAIN_SAMPLES 15
int rainThreshold = 3900;
bool rainDetected = false;
#define DHTPIN 27
#define DHTTYPE DHT11
const int trigPin = 12;
const int echoPin = 13;
```

```
#define SOUND_SPEED 0.034
long duration;
float distanceCm;
const int pH_pin = 35;
float offset = 0.00;
float voltageToPhFactor = 3.5; // Approximate scaling factor
int pin = 32;
const int soilPin = 34; /* Soil moisture sensor O/P pin  ANALOG PIN */
unsigned long lastPhReadTime = 0;
const unsigned long PH_INTERVAL = 604800000UL;
float lastPhValue = 7.0;
bool pumpRunning = false;
unsigned long pumpStartTime = 0;
unsigned long currentPumpTime = 0;
String soilType = "Neutral";  //Default value
int moistureThreshold = 40;   // default %
float pumpFlowRate = 2.0; // liters per minute (change as per pump)
float totalWaterSaved = 0.0; // liters
unsigned long basePumpTime = 5000;

/********************** USER DEPENDENT INFORMATION ***********************/

String weatherApiKey = "571089fbd29c57b0f2adfcf0c97f901a"; //ENTER API KEY OF
WEATHER APP
String city = "Kolkata"; //SELECT CITY
const char* ssid = "realme 5"; //INPUT SSID
const char* password = "162ad1d00865"; //INPUT PASSWORD
float lat = 22.93; // INPUT LATITUDE AND LONGITUDE OF FARM
float lon = 88.38;

/********************** LIBRARIES ***********************/
#include "WiFi.h"
#include <BlynkSimpleEsp32.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include "DHT.h"

/********************** OBJECTS ***********************/
DHT dht(DHTPIN, DHTTYPE);
WidgetTerminal contributorsTerminal(V18);
WidgetTerminal terminal(V9); //FOR FERTILIZER DISPLAY ON BLYNK


/********************** UTILITY FUNCTIONS ***********************/
void calculateFertilizer(float pH) // FERTILIZER RECOMMENDATION BLOCK
{
```

```
  if (pH <= 6.0) {
    fertilizerName = "Agricultural Lime";
    fertilizerBrand = "Tata Agrico / Coromandel Lime";
    fertilizerAmount = 250;  // kg per acre
  }
  else if (pH <= 6.7) {
    fertilizerName = "Dolomite Lime";
    fertilizerBrand = "IPL Dolomite / Rashtriya Chemicals";
    fertilizerAmount = 150;
  }
  else if (pH <= 7.5) {
    fertilizerName = "No fertilizer needed";
    fertilizerBrand = "--";
    fertilizerAmount = 0;
  }
  else if (pH <= 8.0) {
    fertilizerName = "Gypsum / Sulfur";
    fertilizerBrand = "Coromandel Gypsum / Tata Sulphur";
    fertilizerAmount = 100;
  }
  else {
    fertilizerName = "Elemental Sulfur";
    fertilizerBrand = "Zuari Sulphur / IFFCO Sulphur";
    fertilizerAmount = 200;
  }
  fertilizerCalculated = true;
}
void sendToTerminal()
{
  if (offlineMode) return;
  if (!fertilizerCalculated)
  {
    Serial.println("Fertilizer cannot be determined\n");
  }
  terminal.clear();
  String msg = "";
  msg+="\n\n\n\n\n\n\n\n\n";
  msg+="Fertilizer     : " + fertilizerName+"\n";
  msg+="Brand          : " + fertilizerBrand+"\n";
  msg+="Amount         : " + String(fertilizerAmount,1) + " kg/acre";
  terminal.print(msg);
  terminal.flush();
}
void showContributors() //CREDIT BLOCK
{
  if (offlineMode) return;
```

```
  contributorsTerminal.clear();
  contributorsTerminal.println("\n\n");
  contributorsTerminal.println("=== PROJECT CONTRIBUTORS ===");
  contributorsTerminal.println("Mainak Ghosh         Roll no: 16900322015");
  contributorsTerminal.println("Moimon Mandal        Roll no: 16900322016");
  contributorsTerminal.println("Krishanu Adhikari    Roll no: 16900322021");
  contributorsTerminal.println("Debasish Dey         Roll no: 16900322030");
  contributorsTerminal.println("Bishal Das           Roll no: 16900322040");
  contributorsTerminal.println("Project Guide: Dr. Sukanta Bose");
  contributorsTerminal.flush();
}

BLYNK_CONNECTED()
{
 delay(500);

  if (!contributorsShown && !offlineMode)
  {
    showContributors();
    contributorsShown = true;
  }
  sendToTerminal();
}
void updateCropParameters() // SPECIFIC CROP SELECTION LOGIC
{
  if (selectedCrop == "Rice") {
    moistureThreshold = 55;
    basePumpTime = 7000;
    soilType = "Clayey";
  }
  else if (selectedCrop == "Potato") {
    moistureThreshold = 45;
    basePumpTime = 5000;
    soilType = "Loamy";
  }
  else if (selectedCrop == "Wheat") {
    moistureThreshold = 50;
    basePumpTime = 5500;
    soilType = "Loamy";
  }
  else if (selectedCrop == "Onion") {
    moistureThreshold = 43;
    basePumpTime = 4800;
    soilType = "Sandy Loam";
  }
  else if (selectedCrop == "Tomato") {
```

```
    moistureThreshold = 48;
    basePumpTime = 5200;
    soilType = "Loamy";
  }
  else {

    if (lastPhValue < 6.5) moistureThreshold = 45;
    else if (lastPhValue <= 7.5) moistureThreshold = 40;
    else moistureThreshold = 35;
    basePumpTime = 5000;
  }
}
void reportPriorityFault() // FAULT DETECTION DISPLAY
{
    if(!offlineMode)
    {
    if (soilSensorFault)
    {
        Blynk.virtualWrite(V25, "Soil Moisture Error");
    }
    else if (ultrasonicFault)
    {
        Blynk.virtualWrite(V25, "Ultrasonic Error");
    }
    else if (phSensorFault)
    {
        Blynk.virtualWrite(V25, "pH Sensor Error");
    }
    else if (rainSensorFault)
    {
        Blynk.virtualWrite(V25, "Rain Sensor Error");
    }
    else if (dhtFault)
    {
        Blynk.virtualWrite(V25, "DHT11 Error");
    }
    else if (pumpFault)
    {
        Blynk.virtualWrite(V25, "Pump Dry Run Error");
    }
    else
    {
        Blynk.virtualWrite(V25, "All Sensors OK");
    }
    }
}
```

```
void handleConnectivity() {
  // Check WiFi
  if (WiFi.status() != WL_CONNECTED) {
    offlineMode = true;

    if (millis() - lastReconnectAttempt > RECONNECT_INTERVAL) {
      Serial.println("Reconnecting WiFi...");
      WiFi.disconnect();
      WiFi.begin(ssid, password);
      lastReconnectAttempt = millis();
    }
    return; // Skip Blynk if WiFi is down
  }
  // WiFi is OK, check Blynk
  if (!Blynk.connected()) {
    Serial.println("Reconnecting Blynk...");
    if (Blynk.connect(3000)) {
      Serial.println("Blynk reconnected");
      offlineMode = false;
    } else {
      offlineMode = true;
    }
  } else {
    offlineMode = false;
  }
}

/********************** SETUP WIZARD ************************/
void setup() {
updateCropParameters();
analogReadResolution(12);
Serial.begin(115200);
pinMode(pin, OUTPUT);
dht.begin();
pinMode(RAIN_AO_PIN, INPUT);
pinMode(soilPin, INPUT);
pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
pinMode(echoPin, INPUT); // Sets the echoPin as an Input
WiFi.begin(ssid, password);
unsigned long startAttemptTime = millis();
while (WiFi.status() != WL_CONNECTED && millis() - startAttemptTime < 15000) {
//CHANGE IF REQUIRED
  delay(500);
  Serial.println("Connecting to WiFi...");
}
```

```
if (WiFi.status() == WL_CONNECTED) {
Serial.println("Connected to the WiFi network");
Blynk.config(BLYNK_AUTH_TOKEN);
Blynk.connect(3000);
}
else {
  Serial.println("Starting in OFFLINE MODE");
  offlineMode = true;
}
lastWeatherCheck = millis() - WEATHER_INTERVAL;
}

/**********************RAIN SENSOR BLOCK ************************/
void readRainSensor() {

  long total = 0;

  for (int i = 0; i < RAIN_SAMPLES; i++) {
    total += analogRead(RAIN_AO_PIN);
    delay(3);
  }

  int avgRainValue = total / RAIN_SAMPLES;

  Serial.print("Rain Analog Value: ");
  Serial.println(avgRainValue);
  if (avgRainValue <= 5 || avgRainValue >= 4090)
  {
      if (!rainSensorFault)
      {
          if(!offlineMode)
          Blynk.logEvent("sensor_fault", "Rain sensor fault");
          rainSensorFault = true;
      }

      rainDetected = false;
      return;
  }
  else
  {
      rainSensorFault = false;
  }
  if (avgRainValue < rainThreshold) { // Detect rain if value drops below
threshold
    rainDetected = true;
    rainConfirmed = true;
```

```
    rainEventLatched = true;
    rainPredictionStartTime = 0;

    Serial.println(" Rain detected by rain sensor");
  } else {
    rainDetected = false;
    rainConfirmed = false;
  }
}

/********************** pH SENSOR BLOCK ************************/
void updateSoilPh() {
  int sensorValue = analogRead(pH_pin);
  if(sensorValue <= 5 || sensorValue >= 4090)
  {
      if (!phSensorFault)
      {
          if(!offlineMode)
          Blynk.logEvent("sensor_fault", "pH sensor fault");
          phSensorFault = true;
      }
      return;
  }
  else
  {
      phSensorFault = false;
  }
  float voltage = sensorValue * (3.3 / 4095.0);
  lastPhValue = voltageToPhFactor * voltage + offset;
if (selectedCrop == "Others")
{
  if (lastPhValue < 6.5) {
    soilType = "Acidic";
    moistureThreshold = 45;
  }
  else if (lastPhValue <= 7.5) {
    soilType = "Neutral";
    moistureThreshold = 40;
  }
  else {
    soilType = "Basic";
    moistureThreshold = 35;
  }
}
  calculateFertilizer(lastPhValue);
  sendToTerminal();
```

```
  Serial.println("Weekly pH Update:");
  Serial.print("pH = ");
  Serial.println(lastPhValue);
  Serial.print("Soil Type = ");
  Serial.println(soilType);
}

/********************** WEATHER PREDICTION BLOCK ************************/
bool checkRainForecast() {

  Serial.println("Checking weather forecast...");

  HTTPClient http;
  WiFiClient client;
  String url ="http://api.openweathermap.org/data/2.5/forecast?lat="+String(lat,
6)+"&lon="+String(lon, 6)+"&appid="+weatherApiKey+"&units=metric";

  http.begin(client,url);
  int httpCode = http.GET();

  if (httpCode != 200) {
    Serial.print("HTTP Error: ");
    Serial.println(httpCode);
    http.end();
    return false;
  }

  String payload = http.getString();
  http.end();

  StaticJsonDocument<8192> doc;
  DeserializationError error = deserializeJson(doc, payload);

  if (error) {
    Serial.println("JSON parse failed");
    return false;
  }

  bool rainComing = false;

  // Check next 12 hours (4 x 3-hour blocks)
  for (int i = 0; i < 4; i++) {
    float rain = doc["list"][i]["rain"]["3h"] | 0;

    Serial.print("Latitude: ");
  Serial.println(lat);
```

```
  Serial.print("Longitude: ");
  Serial.println(lon);
    Serial.print("Block ");
    Serial.print(i);
    Serial.print(" rain: ");
    Serial.println(rain);

    if (rain > 0) {
      rainComing = true;
      break;
    }
  }

  if (rainComing) {
    Serial.println("Rain predicted in next 12 hours!");
  } else {
    Serial.println(" No rain in next 12 hours.");
  }

  return rainComing;
}

/********************** BLYNK HANDLERS ************************/
BLYNK_WRITE(V6) {
  if (offlineMode) return;
  if (param.asInt() == 1) {
    updateSoilPh();
    lastPhReadTime = millis(); // reset periodic timer

    Blynk.virtualWrite(V7, lastPhValue);
    Blynk.virtualWrite(V8, soilType);
    Blynk.virtualWrite(V14, fertilizerName);
    Blynk.virtualWrite(V15, fertilizerAmount);
    Blynk.virtualWrite(V16, fertilizerBrand);
    sendToTerminal();
    Serial.println("Manual pH + Fertilizer + Brand update");

  }
}
BLYNK_WRITE(V31)
{
  if (offlineMode) return;
  if (param.asInt() == 1)
  {
    selectedCrop = "Rice";
    updateCropParameters();
```

```
    Blynk.virtualWrite(V32, 0);
    Blynk.virtualWrite(V33, 0);
    Blynk.virtualWrite(V34, 0);
    Blynk.virtualWrite(V35, 0);
    Blynk.virtualWrite(V36, 0);

    Serial.println("Selected Crop: Rice");
    Serial.print("Moisture Threshold: ");
    Serial.println(moistureThreshold);
  }
}
BLYNK_WRITE(V32)
{
  if (offlineMode) return;
  if (param.asInt() == 1)
  {
    selectedCrop = "Potato";
    updateCropParameters();

    Blynk.virtualWrite(V31, 0);
    Blynk.virtualWrite(V33, 0);
    Blynk.virtualWrite(V34, 0);
    Blynk.virtualWrite(V35, 0);
    Blynk.virtualWrite(V36, 0);

    Serial.println("Selected Crop: Potato");
    Serial.print("Moisture Threshold: ");
    Serial.println(moistureThreshold);
  }
}
BLYNK_WRITE(V33)
{
  if (offlineMode) return;
  if (param.asInt() == 1)
  {
    selectedCrop = "Wheat";
    updateCropParameters();

    Blynk.virtualWrite(V31, 0);
    Blynk.virtualWrite(V32, 0);
    Blynk.virtualWrite(V34, 0);
    Blynk.virtualWrite(V35, 0);
    Blynk.virtualWrite(V36, 0);

    Serial.println("Selected Crop: Wheat");
```

```
    Serial.print("Moisture Threshold: ");
    Serial.println(moistureThreshold);
  }
}
BLYNK_WRITE(V34)
{
  if (offlineMode) return;
  if (param.asInt() == 1)
  {
    selectedCrop = "Onion";
    updateCropParameters();

    Blynk.virtualWrite(V31, 0);
    Blynk.virtualWrite(V32, 0);
    Blynk.virtualWrite(V33, 0);
    Blynk.virtualWrite(V35, 0);
    Blynk.virtualWrite(V36, 0);

    Serial.println("Selected Crop: Onion");
    Serial.print("Moisture Threshold: ");
    Serial.println(moistureThreshold);
  }
}
BLYNK_WRITE(V35)
{
  if (offlineMode) return;
  if (param.asInt() == 1)
  {
    selectedCrop = "Tomato";
    updateCropParameters();

    Blynk.virtualWrite(V31, 0);
    Blynk.virtualWrite(V32, 0);
    Blynk.virtualWrite(V33, 0);
    Blynk.virtualWrite(V34, 0);
    Blynk.virtualWrite(V36, 0);

    Serial.println("Selected Crop: Tomato");
    Serial.print("Moisture Threshold: ");
    Serial.println(moistureThreshold);
  }
}
BLYNK_WRITE(V36)
{
  if (offlineMode) return;
  if (param.asInt() == 1)
```

```
  {
    selectedCrop = "Others";
    updateCropParameters();

    Blynk.virtualWrite(V31, 0);
    Blynk.virtualWrite(V32, 0);
    Blynk.virtualWrite(V33, 0);
    Blynk.virtualWrite(V34, 0);
    Blynk.virtualWrite(V35, 0);

    Serial.println("Selected Crop: Others");
    Serial.print("Moisture Threshold: ");
    Serial.println(moistureThreshold);
  }
}

/********************** MAIN LOOP **********************/
void loop() {
   handleConnectivity();
  if (!offlineMode) {
    Blynk.run();
  }
  unsigned long now = millis();
bool doTimedLoop = false;
if (now - lastLoopTime >= LOOP_INTERVAL) {
  lastLoopTime = now;
  doTimedLoop = true;
}
/************* RAIN DETECTION AND PREDICTION LOGIC *************/
if(doTimedLoop){
  readRainSensor();
  if (offlineMode) {
  rainPredicted = rainDetected; // fallback to physical sensor in case of no
wifi
}
else if (!offlineMode && millis() - lastWeatherCheck >= WEATHER_INTERVAL) {
  rainPredicted = checkRainForecast();
  lastWeatherCheck = millis();
  rainPredicted = rainPredicted || rainDetected;
  if (rainPredicted && rainPredictionStartTime == 0) {
  rainPredictionStartTime = millis();  // start waiting for rain
}
  Serial.println(rainPredicted ? "Rain predicted → Irrigation will be
skipped":"No rain predicted");
}
else {
```

```
  // Immediate sensor override (no waiting 2 hours)
  if (rainDetected) {
    rainPredicted = true;
  }
}
bool predictionExpired = false;
if (rainPredictionStartTime != 0 && millis() - rainPredictionStartTime >
RAIN_GRACE_PERIOD) {
  predictionExpired = true;
}
if (predictionExpired) {
  rainPredicted = false;
  rainPredictionStartTime = 0;
}
/************** SOIL PH DETECTION *************/
 unsigned long currentMillis = millis();
if (currentMillis - lastPhReadTime >= PH_INTERVAL || lastPhReadTime == 0) {
  updateSoilPh();
  lastPhReadTime = currentMillis;
  if(!offlineMode)
  {
  Blynk.virtualWrite(V7, lastPhValue);
  Blynk.virtualWrite(V8, soilType);
  }
}
/************** ULTRASONIC SENSOR DETECTING TANK WATER LEVEL *************/
digitalWrite(trigPin, LOW);
delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH, 30000); // 30MS TIMEOUT TO PREVENT ESP32 HANG
if (duration == 0)
{
    if (!ultrasonicFault)
    {
        if(!offlineMode)
        Blynk.logEvent("sensor_fault", "Ultrasonic timeout");
        ultrasonicFault = true;
    }

    distanceCm = 100;
}
else
{
    ultrasonicFault = false;
```

```
    distanceCm = duration * SOUND_SPEED / 2;
}
Serial.print("Distance (cm): ");
Serial.println(distanceCm);
int l = constrain(100 - distanceCm, 0, 100);
if(!offlineMode)
Blynk.virtualWrite(V3,l);
if (distanceCm <15) {
if(!offlineMode)
Blynk.virtualWrite(V5, "Tank Has Sufficient Water");
tankEmptyLatched = false;   //  RESET latch when water is present

/************* SOIL MOISTURE DETECTION *************/
int soilMoistureValue = analogRead(soilPin);
if(soilMoistureValue <= 5 || soilMoistureValue >= 4090)
{
    if (!soilSensorFault)
    {
        if(!offlineMode)
        Blynk.logEvent("sensor_fault", "Soil moisture sensor fault");
        soilSensorFault = true;
    }
    digitalWrite(pin, LOW);
    pumpRunning = false;
    reportPriorityFault();
    return;
}
else
{
    soilSensorFault = false;
}
Serial.print("Soil Moisture: ");
Serial.println(soilMoistureValue);
int moisturepercent=map(soilMoistureValue,4095,0,0,100);
int criticalMoisture = moistureThreshold - CRITICAL_MOISTURE_OFFSET;
Blynk.virtualWrite(V4,moisturepercent);
/************* DHT SENSOR READING TEMPERATURE AND HUMIDITY*************/
float h = dht.readHumidity();
float t = dht.readTemperature();
if (isnan(h) || isnan(t))
{
    if (!dhtFault)
    {

        Blynk.logEvent("sensor_fault", "DHT11 sensor fault");
        dhtFault = true;
```

```
    }

    h = 50; //DEFAULT VALUES IN CASE OF FAULT
    t = 25; //DEFAULT VALUES IN CASE OF FAULT
}
else
{
    dhtFault = false;
}

/************** PUMP ON TIME CALCULATION **************/
unsigned long pumpTime = basePumpTime;
float tempFactor = 1.0;
float humFactor = 1.0;
if (t > 25) {
  tempFactor += (t - 25) * 0.03;
}
if (h < 50) {
  humFactor += (50 - h) * 0.02;
}
pumpTime = basePumpTime * tempFactor * humFactor; //PUMPTIME DEPENDS ON BOTH
TEMPERATURE AND HUMIDITY
if (pumpTime > 10000) pumpTime = 10000; //TO PREVENT OVERWATERING
if (pumpTime < 3000) pumpTime = 3000; // TO PREVENT UNDERWATERING

/************** WATER SAVING CALCULATION **************/
if ((rainConfirmed||rainEventLatched) && moisturepercent < moistureThreshold) {
  float waterSaved =(pumpFlowRate / 60.0) * (pumpTime / 1000.0);
  totalWaterSaved += waterSaved;
  Serial.print("Actual rain → Water saved: ");
  Serial.print(waterSaved);
  Serial.println(" L");
  rainConfirmed = false;
  rainEventLatched = false;
}

/************** MAIN IRRIGATION LOGIC **************/
if (rainDetected && !predictionExpired) { // RAIN DETECTED, NO WATERING REQUIRED
  pumpRunning = false;
  digitalWrite(pin, LOW);
}
else if (moisturepercent < criticalMoisture) { //SOIL IS CRITICALLY DRY, WATER
SOIL ANYWAY DESPITE PREDICTION
  Serial.println("⚠ Emergency irrigation override");
  soilWasWet = false;
  rainPredicted = false;
```

```
  rainPredictionStartTime = 0;
  pumpRunning = true;
  pumpStartTime = millis();
  currentPumpTime = pumpTime;
  digitalWrite(pin, HIGH);
}
else if (rainPredicted && !predictionExpired) { // PREDICTION TIMER EXPIRED, SO
WATER ANYWAY
  pumpRunning = false;
  digitalWrite(pin, LOW);
}
else if (moisturepercent < moistureThreshold && soilWasWet && !pumpRunning) {
//ALL CONDITIONS MET FOR WATERING PLANT
  soilWasWet = false;
  Serial.println("Irrigation ON");
  pumpRunning = true;
  pumpStartTime = millis();
  currentPumpTime = pumpTime;
  digitalWrite(pin, HIGH);
}

/************* PUMP SAFETY *************/
if (pumpRunning)
{
    // Dry run protection: tank empty while pump running
    if (distanceCm >= 15)
    {
        if (!pumpFault)
        {
            if(!offlineMode)
            Blynk.logEvent("pump_fault", "Dry run detected");
            pumpFault = true;
        }

        Serial.println("Pump stopped: Tank empty");
        digitalWrite(pin, LOW);
        pumpRunning = false;
    }
    // Normal stop after irrigation complete
    else if (millis() - pumpStartTime >= currentPumpTime)
    {
        Serial.println("Irrigation OFF");
        pumpRunning = false;
        digitalWrite(pin, LOW);

        rainPredicted = false;
```

```
        rainPredictionStartTime = 0;

        pumpFault = false;
    }
}
if (moisturepercent >= moistureThreshold) {
  soilWasWet = true;
  pumpRunning = false;
  digitalWrite(pin, LOW);
}
Serial.print(F("%  Humidity: "));
Serial.print(h);
Serial.print(F("%  Temperature: "));
Serial.print(t);
if(!offlineMode)
{
Blynk.virtualWrite(V1,h);
Blynk.virtualWrite(V0,t);
}
Serial.print(F("°C "));
}
else
{
  pumpFault = false;
Serial.println("Please fill up the tank");
if(!offlineMode)
Blynk.virtualWrite(V5, "TANK IS EMPTY!! REFILL TANK!");
if (!tankEmptyLatched) {                  //  LATCH CHECK
    if(!offlineMode)
    Blynk.logEvent("tank_empty_event", "Tank is empty! Please refill.");
    tankEmptyLatched = true;                //  LATCH SET
  }
digitalWrite(pin, LOW);
}

/************* FINAL UPDATES *************/
if(!offlineMode)
{
Blynk.virtualWrite(V11, totalWaterSaved);     // for total
Blynk.virtualWrite(V12, totalWaterSaved);     // for graph
}
Serial.println("Fertilizer: " + fertilizerName + " | Brand: " + fertilizerBrand
+  " | Amount: " + String(fertilizerAmount));
reportPriorityFault();
}
}
```

Final implementation

This project presents a smart IoT-based irrigation system that automates watering decisions using real-time environmental and soil data. It integrates sensing, prediction, and control to optimize water usage, improve crop health, and ensure reliable operation under both online and offline conditions.

- **Step 1: Initial Irrigation System Implementation**
  The project began with a basic automated irrigation setup using the soil moisture sensor, DHT11, and ultrasonic sensor. At this stage, the system operated on simple logic where only the soil moisture sensor controlled the pump. If the moisture level dropped below a predefined threshold, the pump was turned ON. The ultrasonic sensor acted as a safety mechanism by checking whether the water tank had sufficient water, preventing dry run conditions. The DHT11 sensor was used only for monitoring temperature and humidity, and its data was displayed without influencing irrigation decisions.

- **Step 2: Integration of Soil pH Monitoring**
  The next enhancement involved adding the pH sensor to monitor soil acidity or alkalinity. Initially, this sensor was used only for displaying pH values and did not influence the irrigation logic. However, this step was important as it introduced soil chemistry into the system, laying the groundwork for future intelligent decision-making related to crop health and nutrient management.

- **Step 3: Environmental Awareness using Rain Sensor and Weather API**
  The system was then upgraded to include a rain sensor and weather API integration, making it more environmentally aware. The rain sensor provided real-time rainfall detection, while the weather API enabled prediction of upcoming rainfall. This allowed the system to move beyond reactive irrigation and incorporate predictive control. Irrigation could now be skipped or delayed if rain was detected or expected, improving water efficiency significantly

---

- **Step 4: Intelligent Irrigation Control Logic**
  At this stage, the irrigation logic was significantly improved by combining multiple parameters such as soil moisture, temperature, humidity, pH, and rainfall conditions. The system began deciding not only when to irrigate but also how long to irrigate. Pump runtime became dynamic, adjusting based on environmental conditions like high temperature or low humidity. Emergency conditions, such as critically low moisture, were also handled with override logic to ensure plant safety.

- **Step 5: Water Conservation and Analytics**
  A water-saving mechanism was introduced to quantify the efficiency of the system. Whenever irrigation was skipped due to rainfall, the system calculated the amount of water saved based on pump flow rate and runtime. This value was accumulated over time, providing insight into total water conservation. This feature transformed the system from simple automation into a data-driven solution focused on resource management.

- **Step 6: Fertilizer Recommendation System**
  Using the pH data, the system was enhanced to provide fertilizer recommendations. Based on the detected soil condition, appropriate fertilizers along with their quantities were suggested. This allowed the system to support not only irrigation but also soil nutrient management, ensuring better plant growth and healthier crops.

- **Step 7: User Interaction via Blynk Platform**
  The integration of the Blynk platform enabled real-time monitoring and user interaction. Sensor data such as temperature, humidity, soil moisture, and water level were displayed on a mobile dashboard. Additional features like fertilizer recommendations, contributor display, and water-saving graphs were also implemented. The platform provided remote access and notifications, making the system more user-friendly and interactive.

- **Step 8: Crop-Based Customization**
  To make the system adaptable to different agricultural needs, crop selection functionality was added. Each crop has different water requirements, so selecting a crop automatically adjusts parameters like moisture threshold and irrigation duration. This ensures that the system provides tailored irrigation based on the specific crop being grown.

- **Step 9: Fault Detection and Safety Mechanisms**
  A comprehensive fault detection system was implemented to ensure safe and reliable operation. The system continuously monitors all sensors and identifies abnormal readings or failures. It also includes protection against conditions like pump dry run and sensor faults. When a fault is detected, appropriate actions are taken, such as stopping the pump and notifying the user, ensuring both hardware safety and system reliability.

- **Step 10: Offline Capability and System Resilience**
  The final system was designed to operate even without internet connectivity. In offline mode, the core irrigation logic continues to function using local sensor data, while cloud-based features like weather prediction and remote monitoring are temporarily disabled. The system automatically reconnects when the network becomes available, ensuring uninterrupted operation and high reliability in real-world conditions.

CHAPTER V

Results And Performance Analysis

Table 5.1: Normal operation and empty Tank

Tank Water Level (cm)	Temp. (°C)	Humidity (%)	Rain Detected (%)	Soil Type	Soil Moisture (%)	Pump Condition
70	30	65	9	Neutral	45	OFF
68	32	60	10	Neutral	38	ON
65	31	62	32	Neutral	28	ON (critical)
72	29	70	100	Neutral	50	OFF (rain)
60	33	55	5	Basic	35	ON
75	28	72	0	Neutral	55	OFF
63	34	50	18	Acidic	25	ON (critical)
69	30	66	14	Neutral	42	OFF
58	35	48	5	Basic	33	ON
62	29	68	100	Neutral	46	OFF (rain)
67	31	60	11	Neutral	39	ON
71	27	75	0	Neutral	52	OFF
66	33	58	23	Basic	30	ON (critical)
64	34	52	5	Basic	36	ON
70	28	70	95	Neutral	48	OFF (rain)
68	30	65	7	Neutral	41	OFF
61	35	45	10	Acidic	27	ON (critical)
59	36	42	0	Basic	32	ON
73	29	72	100	Neutral	50	OFF (rain)
74	28	74	5	Neutral	55	OFF
60	34	48	16	Basic	34	ON
62	33	50	14	Basic	29	ON (critical)
65	32	55	5	Neutral	37	ON
69	30	65	100	Neutral	43	OFF (rain)
71	29	68	0	Neutral	49	OFF

Tank Water Level (cm)	Temp. (°C)	Humidity (%)	Rain Detected (%)	Soil Type	Soil Moisture (%)	Pump Condition
58	36	40	11	Acidic	26	ON (critical)
63	34	52	9	Basic	33	ON
67	31	60	5	Neutral	44	OFF
72	28	72	100	Neutral	52	OFF (rain)
70	29	70	0	Neutral	50	OFF
64	33	54	14	Basic	35	ON
61	35	48	18	Acidic	28	ON (critical)
59	36	45	5	Basic	31	ON
73	28	75	100	Neutral	53	OFF (rain)
75	27	78	0	Neutral	56	OFF
62	34	50	14	Basic	34	ON
66	32	55	9	Neutral	38	ON
68	30	60	5	Neutral	42	OFF
71	29	70	100	Neutral	47	OFF (rain)
74	28	74	0	Neutral	52	OFF
60	35	48	18	Acidic	27	ON (critical)
63	34	52	11	Basic	33	ON
67	31	60	5	Neutral	41	OFF
70	30	65	100	Neutral	45	OFF (rain)
72	29	68	0	Neutral	50	OFF
8	33	55	14	Basic	34	OFF (Tank empty)
10	35	48	16	Acidic	39	OFF (Tank empty)
13	36	42	5	Acidic	32	OFF (Tank empty)
14	28	75	9	Neutral	33	OFF (Tank empty)

Tank Water Level (cm)	Temp. (°C)	Humidity (%)	Rain Detected (%)	Soil Type	Soil Moisture (%)	Pump Condition
12	27	78	0	Basic	35	OFF (Tank empty)

The data validates the multi-layered control of the Smart Irrigation System by confirming four key functional aspects:

1. **Normal Operation:** The pump automatically switches ON when Soil Moisture (%) drops below the adaptive threshold set for the Soil Type (e.g., 40% for Neutral soil).
2. **Rain Skip:** High rainfall detection (95–100%) successfully overrides irrigation demand, resulting in an OFF (rain) state to conserve water.
3. **Critical Override:** An ON (critical) state is triggered at extremely low moisture levels (25–30%), ensuring emergency watering to save the crop.
4. **Hardware Safety:** A tank water level of 14 cm or below enforces an OFF (Tank empty) safety shutdown, overriding irrigation requirements to protect the DC pump.

Table 5.2: Pump Condition changes with soil Moisture and Rain

Soil Moisture (%)	Below Critical level	Rain Detected	Rain Predicted (12 hrs.)	Pump Condition
45	No	No	No	OFF
38	No	No	No	ON
28	Yes	No	No	ON (Emergency)
50	No	Yes	Yes	OFF
35	No	No	No	ON
55	No	No	No	OFF
25	Yes	No	Yes	ON(Override)
42	No	No	No	OFF
33	No	No	No	ON
46	No	Yes	Yes	OFF
39	No	No	No	ON
52	No	No	No	OFF

Soil Moisture (%)	Below Critical level	Rain Detected	Rain Predicted (12 hrs.)	Pump Condition
30	Yes	No	No	ON (Emergency)
36	No	No	No	ON
48	No	Yes	Yes	OFF
41	No	No	No	OFF
27	Yes	No	Yes	ON (Override)
32	No	No	No	ON
50	No	Yes	Yes	OFF
55	No	No	No	OFF
34	No	No	No	ON
29	Yes	No	No	ON (Emergency)
37	No	No	No	ON
43	No	Yes	Yes	OFF
49	No	No	No	OFF
33	No	No	No	ON
26	Yes	No	Yes	ON(Override)
44	No	No	No	OFF
52	No	Yes	Yes	OFF
50	No	No	No	OFF
35	No	No	No	ON
28	Yes	No	No	ON (Emergency)
31	No	No	No	ON
53	No	Yes	Yes	OFF
56	No	No	No	OFF
34	No	No	No	ON
38	No	No	No	ON
42	No	No	No	OFF
47	No	Yes	Yes	OFF
52	No	No	No	OFF
27	Yes	No	Yes	ON (Override)
33	No	No	No	ON
41	No	No	No	OFF
45	No	Yes	Yes	OFF

Soil Moisture (%)	Below Critical level	Rain Detected	Rain Predicted (12 hrs.)	Pump Condition
50	No	No	No	OFF
36	No	No	No	ON
29	Yes	No	No	ON (Emergency)
32	No	No	No	ON
53	No	Yes	Yes	OFF
55	No	No	No	OFF

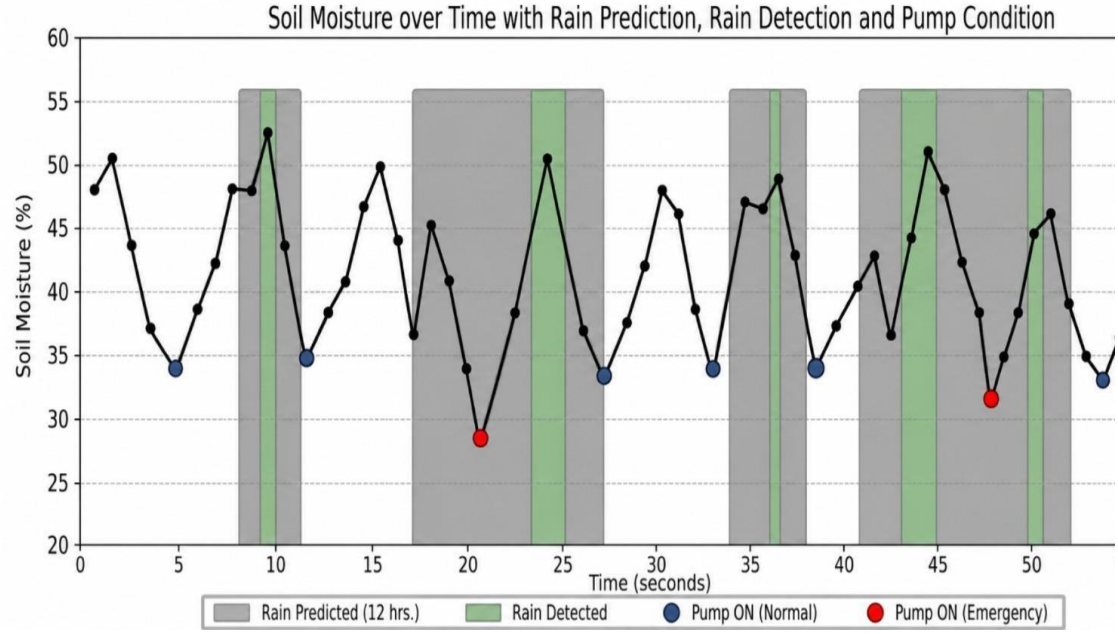

Fig. 5.1: Soil Moisture over Time

Table 5.3: Soil Moisture Threshold changes with PH value

pH Value	Soil Type	Soil Moisture Threshold (%) (Potato)	Soil Moisture (%)	Pump Condition
5.8	Acidic	45	40	ON
6.2	Acidic	45	48	OFF
6.5	Neutral	40	38	ON
7.0	Neutral	40	42	OFF
7.8	Basic	35	30	ON
8.2	Basic	35	36	OFF
5.5	Acidic	45	28	ON

pH Value	Soil Type	Soil Moisture Threshold (%) (Potato)	Soil Moisture (%)	Pump Condition
6.8	Neutral	40	45	OFF
7.6	Basic	35	33	ON
6.9	Neutral	40	39	ON
5.9	Acidic	45	50	OFF
6.7	Neutral	40	37	ON
7.2	Neutral	40	41	OFF
8.0	Basic	35	34	ON
5.6	Acidic	45	27	ON
6.4	Acidic	45	46	OFF
6.6	Neutral	40	35	ON
7.3	Neutral	40	44	OFF
7.9	Basic	35	32	ON
8.3	Basic	35	37	OFF
5.7	Acidic	45	29	ON
6.3	Acidic	45	47	OFF
6.9	Neutral	40	36	ON
7.1	Neutral	40	43	OFF
7.7	Basic	35	31	ON
8.1	Basic	35	38	OFF
5.4	Acidic	45	26	ON
6.5	Neutral	40	39	ON
7.0	Neutral	40	41	OFF
7.8	Basic	35	33	ON
8.4	Basic	35	36	OFF
5.6	Acidic	45	28	ON
6.2	Acidic	45	45	OFF
6.8	Neutral	40	37	ON
7.2	Neutral	40	42	OFF
7.9	Basic	35	30	ON
8.2	Basic	35	35	OFF
5.5	Acidic	45	27	ON
6.4	Acidic	45	46	OFF
6.7	Neutral	40	38	ON

pH Value	Soil Type	Soil Moisture Threshold (%) (Potato)	Soil Moisture (%)	Pump Condition
7.3	Neutral	40	44	OFF
7.6	Basic	35	32	ON
8.0	Basic	35	37	OFF
5.8	Acidic	45	29	ON
6.3	Acidic	45	48	OFF
6.9	Neutral	40	36	ON
7.1	Neutral	40	43	OFF
7.7	Basic	35	31	ON
8.3	Basic	35	38	OFF

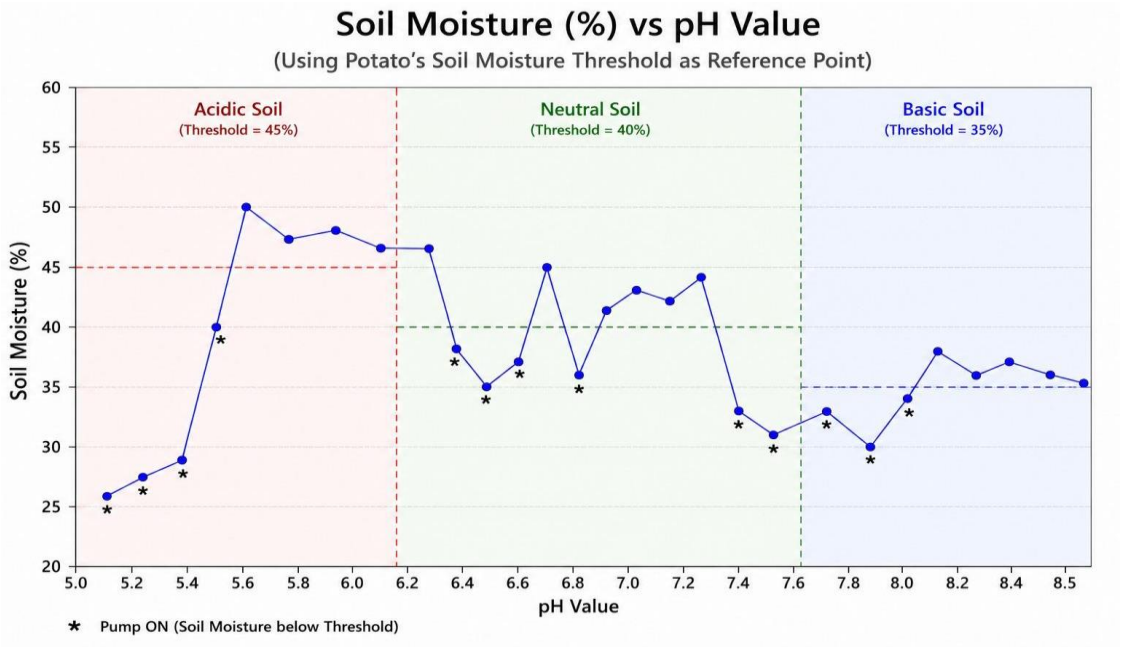

Fig. 5.2: Soil Moisture vs PH of Soil

Table 5.4: Soil Moisture Threshold changes for specific crop

Sl No	Crop Selected	Soil Moisture Threshold (%)	Soil Moisture (%)	Pump Condition
1	Rice	55	30	ON (Emergency)
2	Rice	55	42	ON
3	Rice	55	50	ON
4	Rice	55	56	OFF

Sl No	Crop Selected	Soil Moisture Threshold (%)	Soil Moisture (%)	Pump Condition
5	Rice	55	70	OFF
6	Potato	45	25	ON (Emergency)
7	Potato	45	38	ON
8	Potato	45	44	ON
9	Potato	45	46	OFF
10	Potato	45	60	OFF
11	Wheat	50	32	ON (Emergency)
12	Wheat	50	41	ON
13	Wheat	50	48	ON
14	Wheat	50	52	OFF
15	Wheat	50	65	OFF
16	Onion	43	20	ON (Emergency)
17	Onion	43	35	ON
18	Onion	43	42	ON
19	Onion	43	45	OFF
20	Onion	43	60	OFF
21	Tomato	48	28	ON (Emergency)
22	Tomato	48	40	ON
23	Tomato	48	46	ON
24	Tomato	48	50	OFF
25	Tomato	48	70	OFF
26	Others (Acidic pH<6.5)	45	30	ON (Emergency)
27	Others (Acidic)	45	38	ON
28	Others (Acidic)	45	46	OFF
29	Others (Neutral pH 6.5–7.5)	40	25	ON (Emergency)
30	Others (Neutral)	40	35	ON
31	Others (Neutral)	40	42	OFF
32	Others (Basic pH>7.5)	35	20	ON (Emergency)
33	Others (Basic)	35	30	ON
34	Others (Basic)	35	36	OFF
35	Rice + Rain Detected	55	40	OFF
36	Potato + Rain Predicted	45	35	OFF
37	Wheat + Rain Detected	50	30	OFF
38	Tomato + Rain Predicted	48	42	OFF

Sl No	Crop Selected	Soil Moisture Threshold(%)	Soil Moisture(%)	Pump Condition
39	Onion + Rain Detected	43	25	OFF
40	Rice + Tank Empty	55	20	OFF
41	Potato + Tank Empty	45	30	OFF
42	Wheat + Tank Empty	50	35	OFF
43	Tomato + Tank Empty	48	30	OFF
44	Onion + Tank Empty	43	20	OFF
45	Rice	55	45	ON
46	Potato	45	40	ON
47	Wheat	50	49	ON
48	Tomato	48	47	ON
49	Onion	43	41	ON
50	Others (Neutral)	40	39	ON

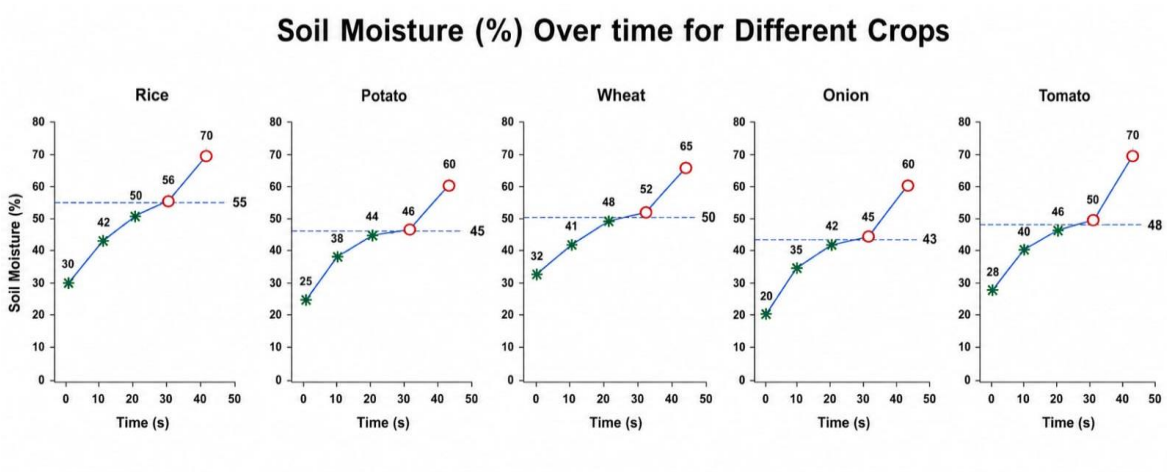

Fig 5.3 Soil Moisture Vs Time for different crops

Table 5.5: Pump ON time changes with Temp. And Humidity

Tank Water Level (cm)	Temperature (°C)	Humidity (%)	Pump ON Time (ms)
70	30	60	5750
68	32	55	6100
65	34	48	7000
72	29	65	5600
60	35	45	7600
75	28	70	5450
63	36	40	8200
69	30	60	5750
58	35	42	7900
62	29	68	5600
67	31	58	5900
71	27	75	5300
66	34	48	7000
64	35	50	6500
70	28	70	5450
68	30	65	5750
61	36	42	8100
59	35	45	7600
73	29	72	5600
74	28	74	5400
60	35	48	7200
62	36	44	8000
65	32	55	6100
69	30	65	5750

Tank Water Level (cm)	Temperature (°C)	Humidity (%)	Pump ON Time (ms)
71	29	68	5600
58	36	40	8200
63	35	48	7200
67	31	60	5900
72	28	72	5450
70	29	70	5500
64	33	54	6300
61	36	42	8100
59	35	45	7600
73	28	75	5400
75	27	78	5200
62	35	50	6500
66	32	55	6100
68	30	60	5750
71	29	70	5500
74	28	74	5400
60	36	42	8100
63	35	48	7200
67	31	60	5900
70	30	65	5750
72	29	68	5600
65	33	55	6200
61	36	44	8000
58	35	45	7600
73	28	75	5400

75	27	78	5200

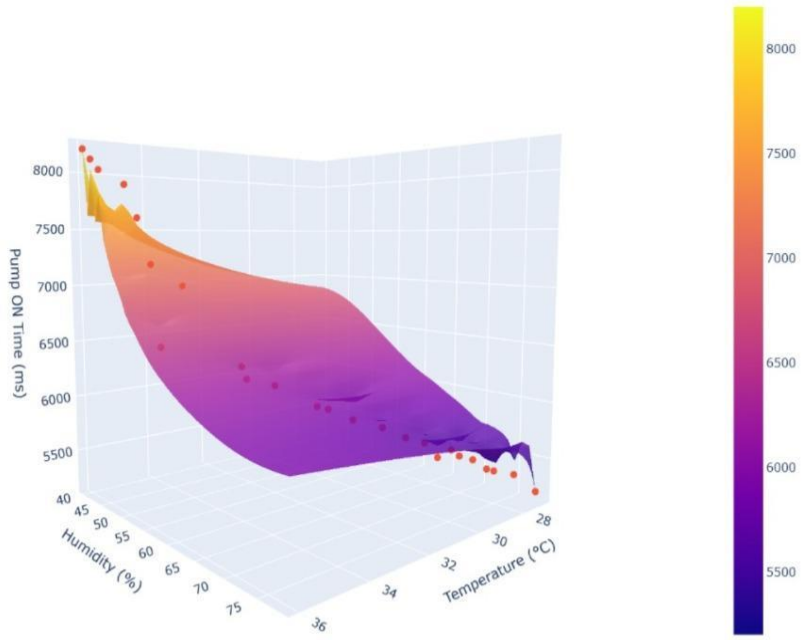

Fig 5.4: Pump On time changes with Temperature and Humidity

Special Features and Uniqueness

1. Adaptive Irrigation:

The system intelligently adjusts irrigation based on real environmental conditions instead of fixed schedules. Soil moisture thresholds change according to soil pH and selected crop, while temperature and humidity dynamically control pump runtime. This ensures efficient water usage and healthier plant growth.

2. Dual Mode Operation:

The system operates in both online and offline modes. When internet is available, it uses a weather API and mobile notifications; during connectivity loss, irrigation continues using local sensor data. This makes the system reliable for rural and low-network areas.

3. Rain-Aware Smart Irrigation:

Irrigation decisions are made using rain sensor data, weather forecasts, and soil moisture conditions together. This "triple-decision" process prevents unnecessary watering, reduces overwatering, and improves water conservation.

4. Automatic Tank Safety and User Alerts:

An ultrasonic sensor continuously monitors tank water level. If the tank becomes empty or water falls below a safe limit, the pump automatically stops to prevent dry running, and the user receives an instant notification through the Blynk app.

5. Emergency Irrigation Override Logic:

If rain is predicted but the soil becomes critically dry, the system waits for a limited period (3 hours) and then overrides the prediction to irrigate the field. This prevents crop damage caused by inaccurate weather forecasts.

6. Soil-Aware Fertilizer Recommendation System:

The system also provides fertilizer recommendations based on real-time soil pH values. Depending on soil condition and selected crop, suitable fertilizers and approximate quantities are displayed on the Blynk mobile app.

Cost Estimation (Prototype)

SL. no.	Components	Qty	Remarks	Cost (Rs.)
1.	ESP 32	1	Microcontroller Unit	250.00
2.	DHT11	1	Humidity and Temperature Sensor	50.00
3.	Soil Moisture Sensor	1	Capacitive Soil Moisture Sensor	60.00
4.	L298N	1	Motor Driver	260.00
5.	pH Sensor	1	pH Sensor Unit	1050.00
6.	Buck Converter	1	DC-DC step-down voltage regulator	50.00
7.	HC-SR04	1	Ultrasonic Sensor	60.00
8.	Pump	1	5v Dc Motor Pump	40.00
9.	Water Tank	1	Split Water Bottle	40.00
10.	DC Battery	2	(9v) Power Supply	50.00
11.	12v Lithium Ion Battery	3	----------	250.00
12.	Bread Board and Connecting Wires	-	----------	150.00
13.	Rain Sensor	1	-----	50.00
			Total	**2360.00**

Table 5.6: Cost Estimation

Applications

1. Smart Farming and Precision Agriculture: In modern agriculture, this system helps farmers make data-driven irrigation decisions by continuously monitoring soil moisture and environmental conditions. By watering only, when necessary, it prevents over-irrigation and under-irrigation, improves crop health, and increases yield while reducing water, electricity, and labour costs.

2. Home Gardening and Terrace Farming: For home gardeners and people with busy lifestyles, the system acts as an automatic plant caretaker. Since watering is based on soil condition rather than timers, plants remain healthy even when the owner is away on vacation. Remote monitoring also allows users to check plant health anytime through the mobile app (Blynk).

3. Greenhouses and Polyhouses: In controlled environments like greenhouses, the system helps maintain ideal growing conditions by considering temperature, humidity, and soil moisture together. This ensures consistent irrigation for high-value crops such as flowers and medicinal plants while reducing the need for manual monitoring.

4. Nurseries and Plant Research Labs: Nurseries and research labs often manage a large number of plants, making manual watering time-consuming. This system automates irrigation while maintaining consistent conditions, which is especially important for plant experiments and research accuracy.

5. Water Conservation Projects: The system supports sustainable farming by reducing water wastage using rain detection and weather prediction. It is particularly useful in drought-prone areas where efficient water usage is essential and can support government or NGO water-saving initiatives.

6. Roadside Plantation and Green Belts: Maintaining roadside plants and highway green belts is difficult due to remote locations. With its offline mode and automatic irrigation, the system ensures continuous watering without manual effort, helping maintain greenery and reduce labour costs.

7. Vertical Farming Systems: Vertical farming requires precise irrigation and environmental monitoring. This system helps maintain optimal growing conditions, reduces water consumption compared to traditional farming, and supports modern urban agriculture.

CHAPTER VI

Future Scope

1. Solar Powered Irrigation

In the future, the system can be powered using solar panels and battery backup, allowing it to run completely off-grid. This would make the solution ideal for remote villages and farms while reducing electricity costs and promoting renewable energy use.

2. Plant Health Monitoring using AI and Camera

Adding a camera module would allow the system to capture images of plant leaves and analyse them using AI. This could help detect diseases, pest attacks, or nutrient deficiencies early and alert farmers before major crop damage occurs.

3. Fertigation Automation

By integrating a nutrient tank and dosing pump, fertilizers could be automatically supplied along with irrigation water. This would reduce manual work and prevent excessive fertilizer usage, making farming more efficient.

4. Machine Learning Based Irrigation

With time, the system can collect historical sensor data and use machine learning to predict irrigation needs automatically. This would allow the system to learn crop patterns and become more intelligent, eventually evolving into a fully autonomous smart farming solution.

Conclusion

The proposed **IoT-based Smart Irrigation System** successfully demonstrates an efficient and intelligent approach for automated watering in both agricultural and gardening applications. By integrating multiple sensors with the ESP32 microcontroller, the system continuously monitors key environmental parameters such as soil moisture, temperature, humidity, water level, rainfall, and soil pH.

Based on real-time sensor data, the system can automatically control irrigation, ensuring that water is supplied only when required. This helps reduce water wastage, minimize manual effort, and maintain suitable soil conditions for healthy plant growth. The inclusion of IoT connectivity through a mobile platform also enables remote monitoring, alerts, and user control, improving overall convenience and system accessibility.

Additional features such as rain detection, tank protection, and fertilizer recommendations further enhance the practicality of the system. Overall, the project highlights the strong potential of combining automation, sensing technology, and cloud connectivity for sustainable resource management. With future enhancements such as predictive analytics and machine learning, the system can become a valuable solution for next-generation smart farming and gardening applications.

Images

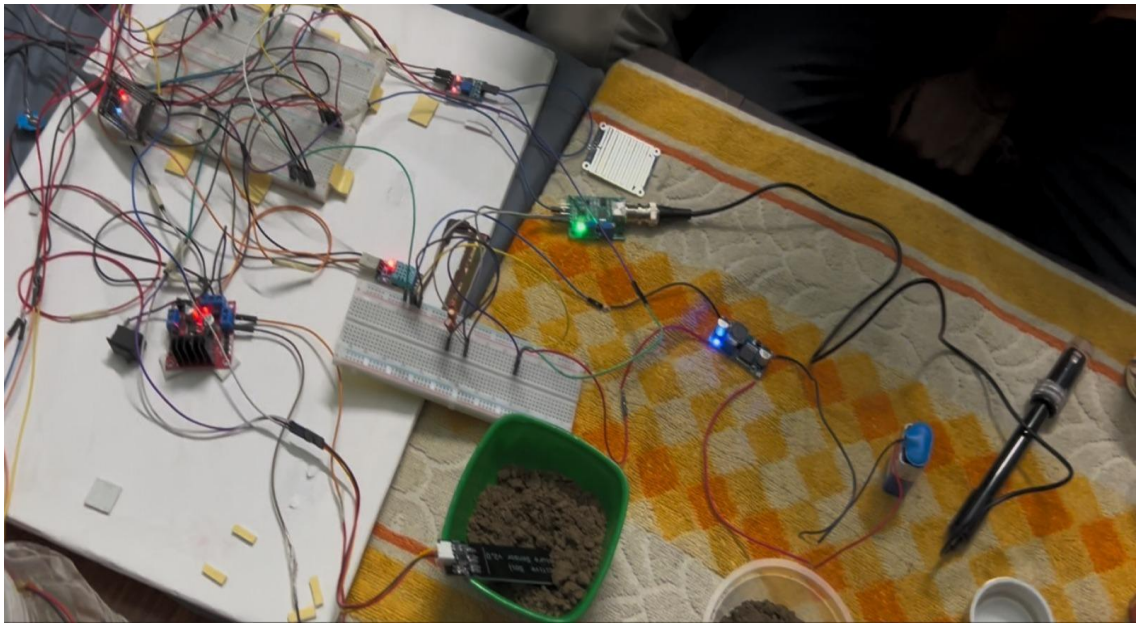

Fig 6.1: System in Action

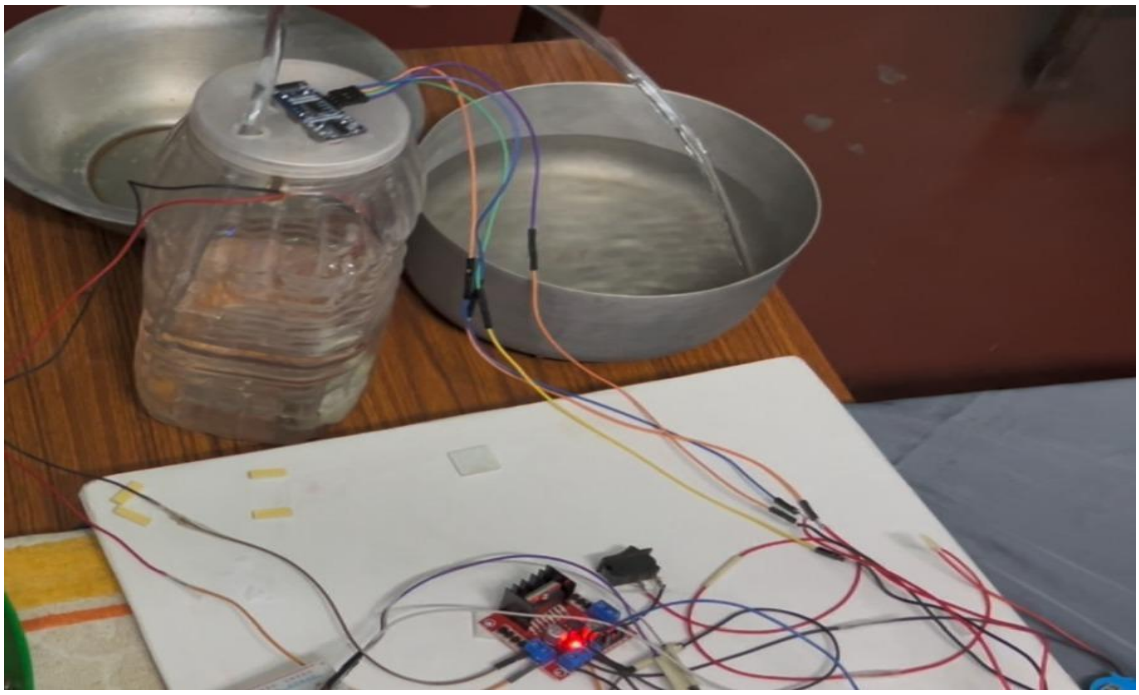

Fig 6.2: Pump extracting water from tank

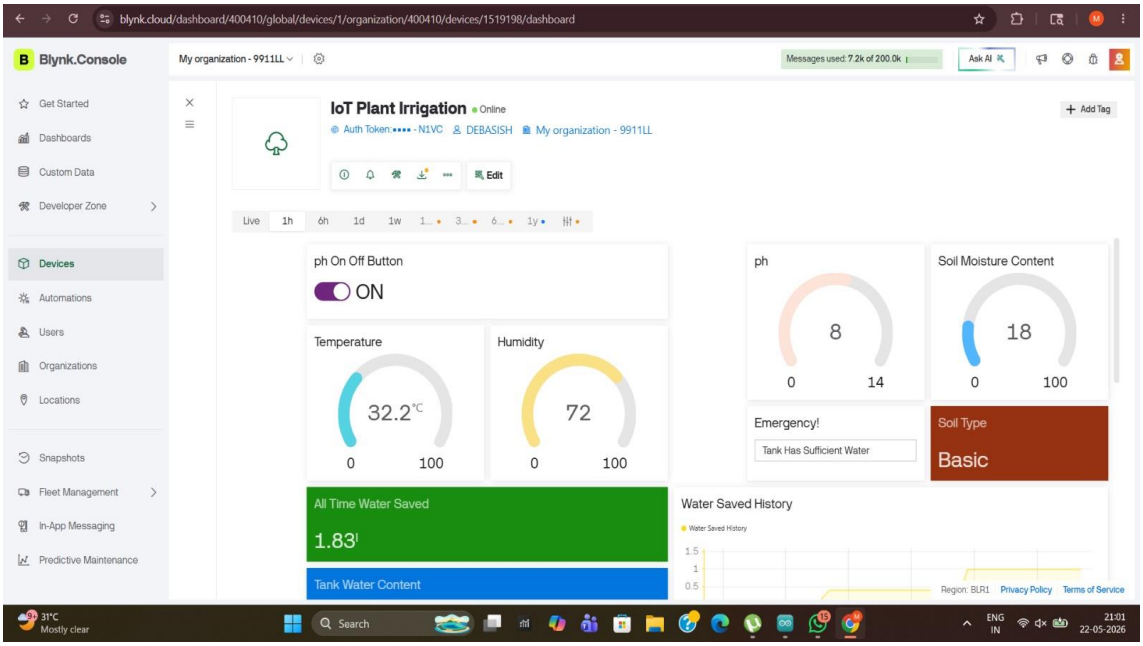

Fig 6.3: Blynk General Display

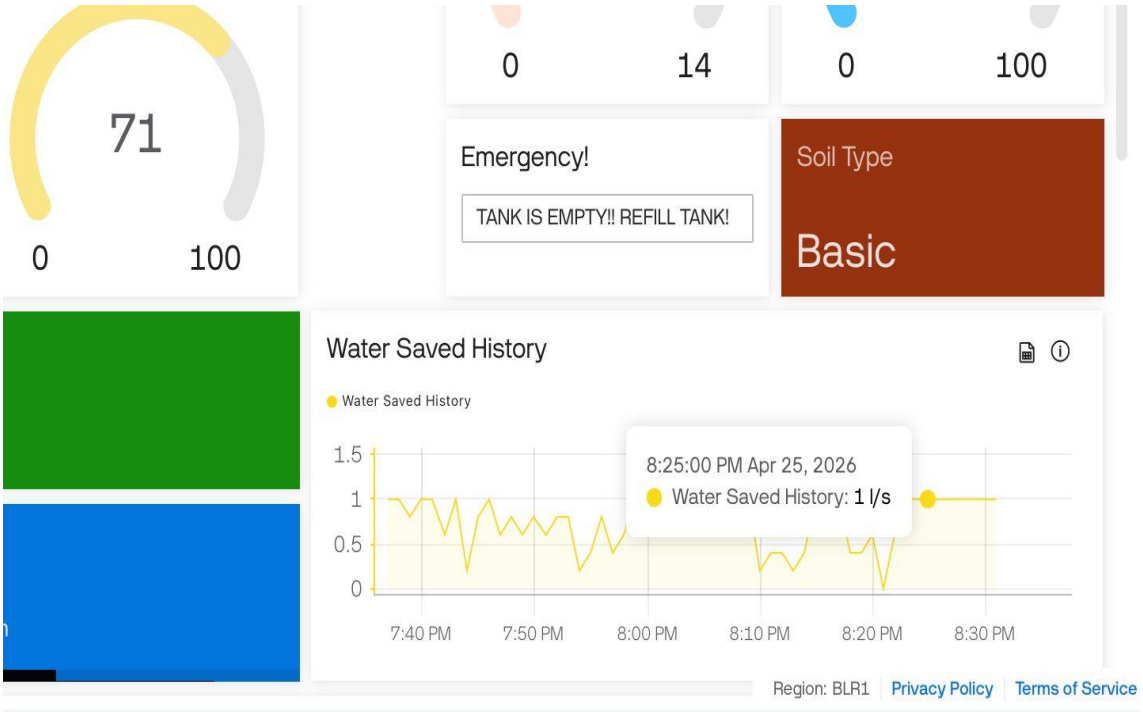

Fig 6.4: Water Saving History

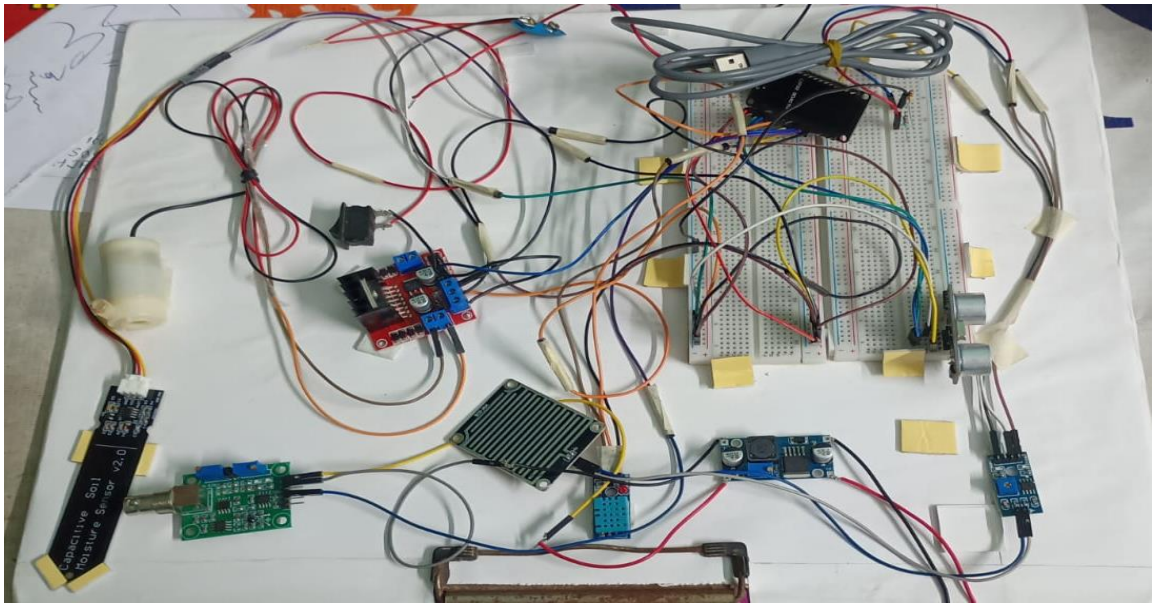

Fig 6.5: Top view of circuit

Fig 6.6: pH sensor being tested in different mediums

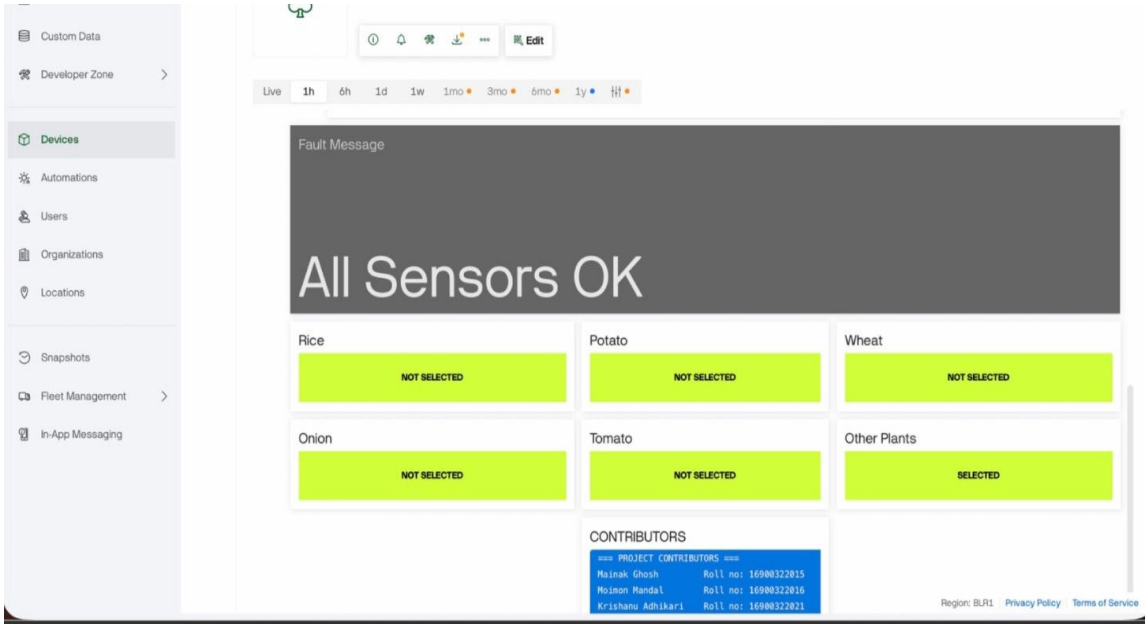

Fig 6.7: Fault detection and crop selection system

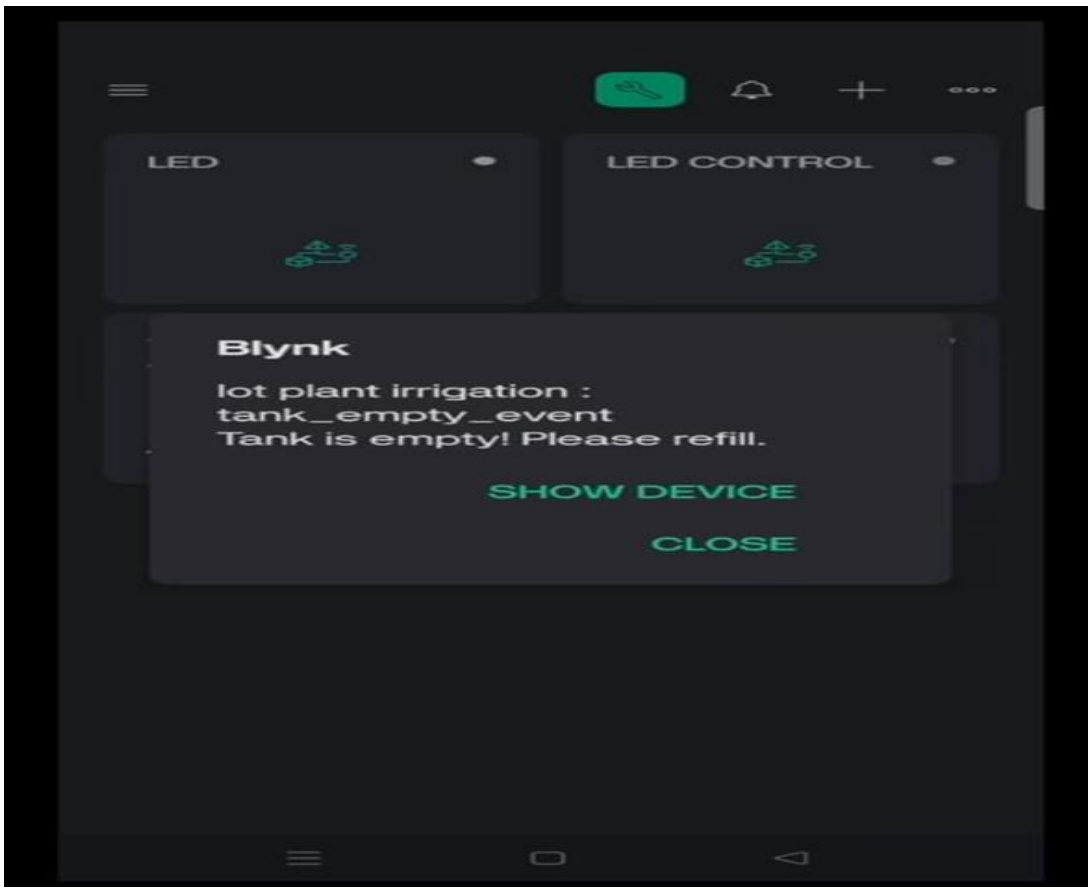

Fig 6.8: Emergency tank empty alert

References

1. Automatic Irrigation System by 555 IC [International Journal of Advances in Engineering and Management (IJAEM) Volume 6, Issue 01 Jan 2024, pp: 598-602 www.ijaem.net ISSN: 2395-5252]
2. Automatic Irrigation System using Moisture Sensor [International Journal of Innovative Research in Electrical, Electronics, Instrumentation and Control Engineering, Vol. 9, DOI:10.17148/IJIREEICE.2021.9557, 05-05-2021]
3. An IoT-based Smart Irrigation System using the Blynk Application [Chetan Naik J. et al. https://share.google/UDg81FZO1AUnlOKFH]]
4. Smart Irrigation System Using Internet of Things [2019 IEEE 9th International Conference on System Engineering and Technology (ICSET), 7 October 2019, Shah Alam, Malaysia]
5. Nor Adni Mat Leh et al."Smart Irrigation System Using Internet of Things"IEEE ICSET 2019.
6. Irmak et al."Irrigation Scheduling Using Climate Data"
7. Pavithra D.S., Srinath M.S., "IoT based Monitoring and Control System for Smart Agriculture," 2015 IEEE Global Conference on Communication Technologies.
8. Allen, R.G., Pereira, L.S., Raes, D., Smith, M. "Crop Evapotranspiration – Guidelines for Computing Crop Water Requirements,
9. Cornell Cooperative Extension, "Soil pH for Field Crops.
10. FAO Irrigation and Drainage Paper 56.
11. Parallax Inc., "Ultrasonic Distance Sensor Principles."
12. Rain Detection System Using Arduino and Rain Sensor: Yogesh, St, Sreedhar TM, Ms.G.T.Bharathy[International Journal of Scientific Research inScience, Engineering and Technology Print ISSN: 2395-1990 |Online ISSN: 2394-4099 (www.ijurset.com)]
13. R. Kamath, N. Balachandran, S. Prabhu,**"Smart Solution for Precision Agriculture using IoT",**2019 International Conference on Communication and Signal Processing (ICCSP).
14. https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2017.00185/full
15. https://www.cambridge.org/core/journals/journal-of-agricultural-science/article/abs/effect-of-shortterm-waterlogging-on-growth-yield-and-nutrient-composition-of-wheat-in-alkaline-soils/F37AAD95A2180D291757068E4C98C121\
16. https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2017.00185/full

17. “DHT11 Humidity and Temperature Sensor Datasheet” - https://components101.com/sites/default/files/component_datasheet/DHT11-Temperature-Sensor.pdf
18. “HC-SR04 Ultrasonic Ranging Module Datasheet” - https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf
19. YL-83 Rain Sensor Module Technical Documentation - https://components101.com/modules/rain-drop-sensor-module
20. “ESP32 Technical Reference Manual” - https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
21. “Capacitive Soil Moisture Sensor v1.2 User Manual” - https://wiki.dfrobot.com/Capacitive_Soil_Moisture_Sensor_SKU_SEN0193
22. L298N Motor Driver Module-L298N Motor Driver Module Pinout, Datasheet, Features & Specs
23. “LM2596 SIMPLE SWITCHER Power Converter Datasheet” - https://www.ti.com/lit/ds/symlink/lm2596.pdf

Appendices

Appendix I: Safety Features Implemented

Safety Feature	Purpose
Dry Run Protection	Prevents pump damage when tank is empty
Sensor Fault Detection	Detects abnormal sensor values
Rain Override	Prevents unnecessary irrigation
Critical Moisture Override	Waters crops during severe dryness
Pump Runtime Limit	Prevents overwatering
Offline Operation	Maintains functionality without internet

Appendix II: Crop-Specific Irrigation Parameters

Crop	Moisture Threshold (%)	Base Pump Time (ms)	Soil Type
Rice	55	7000	Clayey
Potato	45	5000	Loamy
Wheat	50	5500	Loamy
Onion	43	4800	Sandy Loam
Tomato	48	5200	Loamy
Others	Dynamic	5000	Depends on pH

Appendix III: Soil pH Classification and Fertilizer Recommendation

Soil pH Range	Soil Condition	Recommended Fertilizer	Amount(kg/acre)
≤ 6.0	Highly Acidic	Agricultural Lime	250
6.1 – 6.7	Slightly Acidic	Dolomite Lime	150
6.8 – 7.5	Neutral	No Fertilizer Needed	0
7.6 – 8.0	Slightly Basic	Gypsum / Sulfur	100
> 8.0	Highly Basic	Elemental Sulfur	200

Appendix IV: Blynk Virtual Pin Mapping

Virtual Pin	Function
V0	Temperature Display
V1	Humidity Display
V3	Water Tank Level
V4	Soil Moisture Percentage
V5	Tank Status
V6	Manual pH Update Button
V7	pH Value
V8	Soil Type
V9	Fertilizer Recommendation Terminal
V11	Total Water Saved
V12	Water Saved Graph
V14	Fertilizer Name
V15	Fertilizer Amount
V16	Fertilizer Brand
V18	Contributors Terminal
V25	Fault Status Display
V31–V36	Crop Selection Buttons